

fable-engine

Motor de narrativas digitais interativas com exportação offline
e sumarização determinística da estrutura algorítmica

Versão 1.0.0 · Listagem integral do código-fonte

IDENTIFICAÇÃO DO PROGRAMA

Título	fable-engine
Versão	1.0.0
Linguagem	TypeScript (compilado para JavaScript ECMAScript 2020)
Campo de aplicação	Educação; tecnologia educacional; informática na educação
Tipo de programa	Biblioteca de software (motor de execução) com aplicação cliente em Canvas 2D
Data de criação	02/07/2026 (primeiro registro no repositório de controle de versão)
Data desta versão	10/07/2026
Instituição	Programa de Pós-Graduação em Informática na Educação (PPGIE), Universidade Federal do Rio Grande do Sul (UFRGS)
Licença de uso	MIT
Repositório	https://github.com/RandMelville/fable-engine
DOI (Zenodo)	10.5281/zenodo.21302155

AUTORES

- Randerson Oliveira Melville Rebouças:** autoria técnica (concepção e implementação do motor).
ORCID 0009-0005-3056-5074 · cota de participação declarada: 60%
- Marcelo Magalhães Foohs:** autoria pedagógica (concepção do Ciclo de Remediação Ativa e do desenho pedagógico).
ORCID 0000-0002-4735-0732 · cota de participação declarada: 25%
- Rosa Maria Vicari:** coorientação científica.
ORCID 0000-0002-6909-6405 · cota de participação declarada: 15%

RESUMO DESCRITIVO DO PROGRAMA

O fable-engine é um motor de narrativas digitais interativas, escrito em TypeScript e sem dependências externas de execução, que executa histórias ramificadas em Canvas 2D no próprio navegador. Seu modelo de dados (cenar, variáveis de estado, escolhas e condicionais) representa uma narrativa interativa como uma máquina de estados e resume, de forma determinística, as construções computacionais que a compõem (cenar, variáveis, ramificações e modificadores). O motor exporta cada narrativa como um arquivo HTML autocontido que roda inteiramente offline, sem servidores nem requisições de rede. Foi desenvolvido no contexto da plataforma educacional Remidição (PPGIE/UFRGS), para operacionalizar a remediação de textos escritos em histórias jogáveis, e é publicado como biblioteca de código aberto reutilizável sob licença MIT.

IDENTIFICAÇÃO DIGITAL DO CÓDIGO-FONTE

O resumo digital abaixo foi calculado com o algoritmo **SHA-512** sobre a concatenação, em ordem alfabética de caminho, dos pares *caminho do arquivo* e *conteúdo do arquivo* de todos os 25 arquivos transcritos neste documento, separados pelo octeto nulo (formato: `caminho\0conteúdo\0`), com os arquivos codificados em UTF-8 e terminadores de linha LF. O cálculo é reproduzível por qualquer terceiro a partir da tag `v1.0.0` do repositório público.

SHA-512: 5522e9c4 4331bd13 e2a5812f f12b3d5f e9d6f550 93a99a37 24a87132 ad9722eb 18e32099 7cf94aed f79e82e8 4af4ee62 a2834a28 5dc4e87a 5e7279e5 bb66c86f

Commit (SHA-1): b4eeb77b0332fc716017a7f36de915102c228dc2

MANIFESTO DE ARQUIVOS

#	Arquivo	Linguagem	Linhas	Bytes	SHA-256
1	.gitignore	texto	6	138	f324ccf6a1c56556066cc00e5aba8e312dd6e7d52458ad60bf1969f900a35c40
2	.zenodo.json	JSON	32	1.479	6af73d4a78cde3cd0b9445ed15ddf06e2acff510b3c2155e9c3433b32bef5459
3	CITATION.cff	YAML (CITATION.cff)	38	2.063	c8372dacaf8c5503cb374670d3392628c794a6c1142551e96acea37873405f79
4	LICENSE		21	1.094	f561adb6f3c8716860c70442d14a1bc89d873c612bc8731573aa45c153aae24c
5	README.md	Markdown	114	4.800	7bcf65adaf5da15021c3f3f128ec529dea95276962528651372af407609ea5c0
6	examples/fox-and-wolf.ts	TypeScript	192	5.021	5cc8d87b3adbd3a7d2b62fdaa6d822eb6a2abf6bf94d5a5b588cd7cfe99b8ae
7	package-lock.json (<i>não transcrito</i>)	JSON	509	14.866	c1fdc727192dc86d1a100deeda53caae951c790f31f69aca6d0ea7b5396f23e1
8	package.json	JSON	52	1.567	42de962075f9fe347007a4968fa4b2672e33c180e77352dc2895e3388615c66c
9	scripts/build-standalone.mjs	JavaScript (ESM)	27	917	1c9fc04fb9c0de5532fd8bab3bbef0e1398326582048979bc5967b592f5971aa
10	scripts/run-selfcheck.mjs	JavaScript (ESM)	27	877	a3be085eb71638caf26b1f4c3731b07924c90efb70abf2d6db88a89b71c3f6e5
11	src/avatars.ts	TypeScript	34	1.192	4b58aefefdef605b35f68fe07efe80cd80f71a2e9aa5892103d33442a46d15cc
12	src/backgrounds.ts	TypeScript	26	1.110	fe5d6ba8679baf12b82b7df1a21ba9aa39bf5ffbe34399d389f02ca928104e1e
13	src/drawBackground.ts	TypeScript	189	4.924	9aac88a87d8a50aabe110f1aa0e82a80924f00cf99573d310b99218969d2624a
14	src/drawCharacter.ts	TypeScript	240	6.632	808a644a35ec383c207e3d23396128a918102ce7cc6d6c3bd387c47901451455
15	src/engine.ts	TypeScript	224	6.282	68b2c5813841e80afaf4de59f91c6fddd0e28cd9376bdc9bdbc24eb7fd754a7
16	src/export.ts	TypeScript	89	3.498	c67d3e7cbb3821c08d1874fe26b6c0f1a184f096d3359811ace0d53ff599bb90
17	src/generated/standaloneBundle.ts	TypeScript	2	13.007	5ff05c9bc2d3dcb4f783a2cff026f3abaa993563a368b876b05956dd47fb9d15
18	src/index.ts	TypeScript	32	561	3682fc8b5bc18646e68f6ea307efd5cc92c515a6ef2f588b66f6cb564e571fe6

#	Arquivo	Linguagem	Linhas	Bytes	SHA-256
19	src/moods.ts	TypeScript	24	828	404ccc27448d44d33253ec47c1538497617887a5d0ab7295d9f96ee86ffa7fbf
20	src/movements.ts	TypeScript	50	1.148	d9af275cfe2bc4ede02d3e4c9e147022141d615332a46aabc01599059e08afaf
21	src/player.ts	TypeScript	430	13.213	58cfcb3a43a035cab0b7954fcf28c951fd6052fe6b994d2660bf7483c0a4435f
22	src/standalone.ts	TypeScript	22	782	0326f717e51f40d040d6a16d4a310fc28452ede88cb17cd67852c56e7f7030c1
23	test/e2e.mjs	JavaScript (ESM)	135	4.435	ca7b77adea73fb72e593ac3cb6ab140a820f7ad3e8c8976e00e46cfc06769033
24	test/selfcheck.ts	TypeScript	92	2.882	6bc02d416c5e168e7015cfa680a5d52e7cdd28d0e0b4e8ae64f862b5f6dc87a0
25	tsconfig.build.json	JSON	8	128	a42408a1fe307dae1330403900eed592a1d100893d0011a2197773df677b631e
26	tsconfig.json	JSON	18	444	94c33ce16453a0c80d8c3b806660d50d96c04ea2650deaeab2d47b06e8103e6b

Nota. O arquivo `package-lock.json` consta do manifesto, com seu respectivo resumo digital, mas não é transcrito na listagem: trata-se de arquivo gerado automaticamente pelo gerenciador de pacotes npm, contendo apenas metadados de versões e endereços das dependências de desenvolvimento, sem código autoral. Não integra, portanto, o conjunto sobre o qual foi calculado o SHA-512 acima. O arquivo `src/generated/standaloneBundle.ts` é transcrito, embora seja artefato gerado pelo processo de compilação a partir de `src/standalone.ts`, por ser distribuído junto ao código-fonte.

.gitignore

Arquivo 1 de 25 · texto · 6 linhas · 138 bytes

SHA-256: f324ccf6a1c56556066cc00e5aba8e312dd6e7d52458ad60bf1969f900a35c40

```
1 | node_modules/  
2 | dist/  
3 |  
4 | # Documentos internos de registro/autoria (INPI, CPFs, taxas, política de PI) – não publicar  
5 | registro/  
6 | .DS_Store
```

fim de .gitignore

.zenodo.json

Arquivo 2 de 25 · JSON · 32 linhas · 1.479 bytes

SHA-256: 6af73d4a78cde3cd0b9445ed15ddf06e2acff510b3c2155e9c3433b32bef5459

```
1  {
2    "title": "fable-engine: a framework-free engine for interactive narratives with offline export and
algorithmic-structure summary",
3    "description": "A tiny, dependency-free TypeScript engine for interactive branching narratives. It
plays stories in a Canvas 2D player, exports each story as a single self-contained offline HTML file
(file over app), and deterministically summarizes the algorithmic structure of each narrative (scenes,
variables, branches, modifiers) as evidence for interpreting computational thinking. Developed within the
RemidiAção platform (PPGIE/UFRGS) to operationalize the remediation of written texts into interactive
digital narratives.",
4    "upload_type": "software",
5    "license": "MIT",
6    "access_right": "open",
7    "keywords": [
8      "interactive narrative",
9      "narrative engine",
10     "computational thinking",
11     "algorithmic structure",
12     "educational technology",
13     "file over app"
14   ],
15   "creators": [
16     {
17       "name": "Rebouças, Randerson Oliveira Melville",
18       "affiliation": "PPGIE, Universidade Federal do Rio Grande do Sul (UFRGS)",
19       "orcid": "0009-0005-3056-5074"
20     },
21     {
22       "name": "Foohs, Marcelo Magalhães",
23       "affiliation": "PPGIE, Universidade Federal do Rio Grande do Sul (UFRGS)",
24       "orcid": "0000-0002-4735-0732"
25     },
26     {
27       "name": "Vicari, Rosa Maria",
28       "affiliation": "PPGIE, Universidade Federal do Rio Grande do Sul (UFRGS)",
29       "orcid": "0000-0002-6909-6405"
30     }
31   ]
32 }
```

fim de .zenodo.json

CITATION.cff

Arquivo 3 de 25 · YAML (CITATION.cff) · 38 linhas · 2.063 bytes

SHA-256: c8372dacaf8c5503cb374670d3392628c794a6c1142551e96acea37873405f79

```

1  cff-version: 1.2.0
2  message: "If you use this software, please cite it as below."
3  title: "fable-engine: a framework-free engine for interactive narratives with offline export and
    algorithmic-structure summary"
4  abstract: >-
5      A tiny, dependency-free TypeScript engine for interactive branching narratives.
6      It plays stories in a Canvas 2D player, exports each story as a single self-contained
7      offline HTML file (file over app), and deterministically summarizes the algorithmic
8      structure of each narrative (scenes, variables, branches, modifiers) as evidence for
9      interpreting computational thinking. Developed within the RemidiAção platform
10     (PPGIE/UFRGS) to operationalize the remediation of written texts into interactive digital
11     narratives.
12  type: software
13  version: 1.0.0
14  license: MIT
15  repository-code: "https://github.com/RandMelville/fable-engine"
16  keywords:
17      - interactive narrative
18      - narrative engine
19      - computational thinking
20      - algorithmic structure
21      - educational technology
22      - file over app
23  authors:
24      # Randerson – autoria técnica (desenho e implementação do motor)
25      - family-names: "Rebouças"
26        given-names: "Randerson Oliveira Melville"
27        affiliation: "Programa de Pós-Graduação em Informática na Educação (PPGIE), Universidade Federal do
    Rio Grande do Sul (UFRGS)"
28        orcid: "https://orcid.org/0009-0005-3056-5074"
29      # Marcelo – autoria pedagógica (concepção do CRA e do desenho pedagógico que o motor operacionaliza)
30      - family-names: "Foohs"
31        given-names: "Marcelo Magalhães"
32        affiliation: "Programa de Pós-Graduação em Informática na Educação (PPGIE), Universidade Federal do
    Rio Grande do Sul (UFRGS)"
33        orcid: "https://orcid.org/0000-0002-4735-0732"
34      # Rosa – orientação científica (o motor deriva da tese de doutorado que ela coorienta)
35      - family-names: "Vicari"
36        given-names: "Rosa Maria"
37        affiliation: "Programa de Pós-Graduação em Informática na Educação (PPGIE), Universidade Federal do
    Rio Grande do Sul (UFRGS)"
38        orcid: "https://orcid.org/0000-0002-6909-6405"

```

fim de CITATION.cff

LICENSE

Arquivo 4 de 25 · · 21 linhas · 1.094 bytes
SHA-256: f561adb6f3c8716860c70442d14a1bc89d873c612bc8731573aa45c153aae24c

```
1  MIT License
2
3  Copyright (c) 2026 Randerson Oliveira Melville Rebouças
4
5  Permission is hereby granted, free of charge, to any person obtaining a copy
6  of this software and associated documentation files (the "Software"), to deal
7  in the Software without restriction, including without limitation the rights
8  to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
9  copies of the Software, and to permit persons to whom the Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be included in all
13 copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
21 SOFTWARE.
```

fim de LICENSE

README.md

Arquivo 5 de 25 · Markdown · 114 linhas · 4.800 bytes

SHA-256: 7bcf65adaf5da15021c3f3f128ec529dea95276962528651372af407609ea5c0

```

1  # fable-engine
2
3  A tiny, framework-free TypeScript engine for interactive narratives, with a Canvas 2D
4  player and self-contained, offline HTML export. Zero runtime dependencies.
5
6  You describe a story as data (scenes, state variables, choices, conditionals); the engine plays
7  it in the browser and can export it as a single .html file that runs anywhere, offline,
8  forever, with no server and no network requests.
9
10 It was built to make the algorithm the author builds observable and executable: the story is a
11 state machine the player runs, and summarizeAlgorithmicStructure gives a deterministic account of
12 the computational constructs in it (scenes, variables, branches, modifiers), a description of what
13 was built as evidence for interpreting computational thinking, not a score of cognitive faculties.
14
15 ## Install
16
17 ```bash
18 npm install fable-engine
19 ```
20
21 ## Quick start
22
23 ```ts
24 import { Player, exportHtml, summarizeAlgorithmicStructure, type Story } from "fable-engine";
25
26 const story: Story = {
27   formatVersion: 1,
28   title: "The Fox and the Wolf",
29   author: "you",
30   variables: [{ id: "courage", name: "courage", type: "number", initialValue: 0 }],
31   initialSceneId: "s1",
32   scenes: [
33     {
34       id: "s1",
35       title: "At the forest's edge",
36       characters: [{ id: "fox", name: "Fox", avatar: "cat", color: "#EA580C", x: 120, y: 250 }],
37       actions: [
38         { type: "appear", characterId: "fox" },
39         { type: "speak", characterId: "fox", text: "A choice awaits.", durationMs: 1600 },
40       ],
41       choices: [
42         { id: "brave", buttonLabel: "Step forward", targetSceneId: "s2",
43           modifiers: [{ variableId: "courage", operation: "+", value: 1 }] },
44         { id: "wary", buttonLabel: "Hang back", targetSceneId: "s2", modifiers: [] },
45       ],
46     },
47     { id: "s2", title: "The clearing", characters: [], actions: [] }, // ending scene
48   ],
49 };
50
51 // Play it on the page
52 new Player(document.getElementById("stage")!, story);
53
54 // See a deterministic summary of this story's computational structure
55 summarizeAlgorithmicStructure(story);
56 // → { scenes: 2, variables: 1, branches: 2, modifiers: 1 }
57
58 // Or export a self-contained, offline .html
59 const html = exportHtml(story);
60 ```
61
62 > `Conditional` (state-based branching) and `repeat` (loops) are shown in the full runnable
63 > story in [examples/fox-and-wolf.ts](./examples/fox-and-wolf.ts).
64
65 ## Why it exists
66
67 - Offline & portable (*file over app*). exportHtml inlines the story JSON, the player

```



```

68     runtime, and the CSS into one HTML file with **no external references**; it opens offline in
69     any browser, with no platform behind it.
70 - **A small, honest vocabulary.** The whole surface is `Scene`, `Variable`, `Choice`,
71     `Conditional`, and nothing else to learn that isn't also a decision about the story.
72 - **Instrumentable.** The player tracks the path taken and the final state, so the narrative
73     itself becomes research data.
74 - **Deterministic structural summary.** `summarizeAlgorithmicStructure` describes the computational
75     constructs in a story (scenes, variables, branches, modifiers) by counting: reproducible, no AI,
76     no inference. It describes what the author built, not a cognitive score.
77
78 ## Public API
79
80 | Export | Purpose |
81 | --- | --- |
82 | `Story`, `Scene`, `Variable`, `Choice`, `Conditional`, `Action`, `Modifier`, `Character`, `State` | The
data model (types). |
83 | `Player(container, story, options?)` | Canvas 2D player. |
84 | `exportHtml(story, options?)`, `downloadHtml(name, html)` | Self-contained offline export. |
85 | `initialState`, `evaluateConditional`, `applyModifier`, `nextSceneByConditional`, `flatten` | Pure
engine functions (fully unit-tested). |
86 | `summarizeAlgorithmicStructure(story)` | Deterministic summary of a story's computational structure. |
87
88 ## Develop
89
90 ```bash
91 npm run build      # standalone bundle + dist (ESM + .d.ts)
92 npm test           # headless self-check (engine unit tests)
93 npm run test:e2e   # headless-browser check: exported .html plays offline, zero network requests
94 npm run typecheck  # strict TypeScript, no `any`
95 ```
96
97 ## License
98
99 [MIT](./LICENSE).
100
101 ## Citing
102
103 If you use fable-engine in academic work, please cite it (see [CITATION.cff](./CITATION.cff)).
104
105 ---
106
107 ### Contexto (pt-BR)
108
109 O `fable-engine` nasceu na plataforma educacional **Remediação** (PPGIE/UFRGS), onde
110 operacionaliza a **remediação** de textos escritos em narrativas digitais interativas, parte do
111 **Ciclo de Remediação Ativa (CRA)**. Aqui ele é publicado como biblioteca genérica e reutilizável;
112 o vocabulário de autoria (cena, variável, escolha, condicional) e o resumo determinístico da
113 estrutura são os mesmos, com a API em inglês para maior alcance. A API pública é `Story` (a "fábula");
114 você a descreve
115 como dado e o motor a executa e exporta offline.

```

fim de README.md

examples/fox-and-wolf.ts

Arquivo 6 de 25 · TypeScript · 192 linhas · 5.021 bytes

SHA-256: 5cc8d87b3adbd3a7d2b62fdaa6d822eb6a2abf6bfff94d5a5b588cd7cfe99b8ae

```

1  import type { Story } from "../src/engine";
2
3  /**
4   * Example story – "The Fox and the Wolf".
5   *
6   * Deliberately exercises every engine feature:
7   * – appear/speak/move      (sprite, say, animation)
8   * – repeat                 (replays visual movement, an animation step, not an authored loop)
9   * – variable `courage`    (declared state)
10  * – choice with modifier   (interactive branch + state effect)
11  * – conditional            (state-based branch)
12  */
13  export const foxAndWolf: Story = {
14    formatVersion: 1,
15    title: "The Fox and the Wolf",
16    author: "example-author",
17    variables: [
18      { id: "courage", name: "courage", type: "number", initialValue: 0 },
19    ],
20    initialSceneId: "c1",
21    scenes: [
22      {
23        id: "c1",
24        title: "At the forest's edge",
25        text: "The Fox comes hopping to the edge of the forest.",
26        characters: [
27          {
28            id: "fox",
29            name: "Fox",
30            avatar: "cat",
31            color: "#EA580C",
32            x: 120,
33            y: 250,
34          },
35        ],
36        actions: [
37          { type: "appear", characterId: "fox" },
38          {
39            type: "speak",
40            characterId: "fox",
41            text: "What a lovely day for an adventure!",
42            durationMs: 1800,
43          },
44          // loop: the fox takes 3 little hops (algorithmic thinking made visible)
45          {
46            type: "repeat",
47            times: 3,
48            actions: [
49              { type: "move", characterId: "fox", dx: 60, dy: -40, durationMs: 350 },
50              { type: "move", characterId: "fox", dx: 60, dy: 40, durationMs: 350 },
51            ],
52          },
53          {
54            type: "speak",
55            characterId: "fox",
56            text: "Oops... I heard a noise over there!",
57            durationMs: 1800,
58          },
59        ],
60        choices: [
61          {
62            id: "e1",
63            buttonLabel: "Face whatever is coming",
64            targetSceneId: "c2",
65            modifiers: [{ variableId: "courage", operation: "+", value: 2 }],
66          },
67        ],

```

```

68         id: "e2",
69         buttonLabel: "Hide behind the tree",
70         targetSceneId: "c2",
71         modifiers: [{ variableId: "courage", operation: "-", value: 1 }],
72     },
73 ],
74 },
75 {
76     id: "c2",
77     title: "The encounter",
78     text: "Out of the shadows steps the Wolf.",
79     characters: [
80         {
81             id: "fox",
82             name: "Fox",
83             avatar: "cat",
84             color: "#EA580C",
85             x: 150,
86             y: 250,
87         },
88         {
89             id: "wolf",
90             name: "Wolf",
91             avatar: "bear",
92             color: "#475569",
93             x: 560,
94             y: 250,
95         },
96     ],
97     actions: [
98         { type: "appear", characterId: "fox" },
99         { type: "appear", characterId: "wolf" },
100         {
101             type: "speak",
102             characterId: "wolf",
103             text: "Who dares enter my forest?",
104             durationMs: 2000,
105         },
106         { type: "move", characterId: "wolf", dx: -120, dy: 0, durationMs: 900 },
107     ],
108     // branches by the state accumulated in the choices (algorithmic thinking)
109     conditional: {
110         variableId: "courage",
111         operator: ">=",
112         value: 2,
113         sceneIfTrueId: "c_victory",
114         sceneIfFalseId: "c_flight",
115     },
116 },
117 {
118     id: "c_victory",
119     title: "Courage!",
120     text: "With courage, the Fox faces the Wolf.",
121     characters: [
122         {
123             id: "fox",
124             name: "Fox",
125             avatar: "cat",
126             color: "#EA580C",
127             x: 200,
128             y: 250,
129         },
130         {
131             id: "wolf",
132             name: "Wolf",
133             avatar: "bear",
134             color: "#475569",
135             x: 440,
136             y: 250,
137         },
138     ],
139     actions: [

```

```
140     { type: "appear", characterId: "fox" },
141     { type: "appear", characterId: "wolf" },
142     {
143       type: "speak",
144       characterId: "fox",
145       text: "I am not afraid of you!",
146       durationMs: 2000,
147     },
148     { type: "move", characterId: "wolf", dx: 220, dy: 0, durationMs: 1100 },
149     {
150       type: "speak",
151       characterId: "fox",
152       text: "The forest belongs to all of us! 🐾",
153       durationMs: 2200,
154     },
155   ],
156 },
157 {
158   id: "c_flight",
159   title: "The escape",
160   text: "Without enough courage, the Fox runs away.",
161   characters: [
162     {
163       id: "fox",
164       name: "Fox",
165       avatar: "cat",
166       color: "#EA580C",
167       x: 400,
168       y: 250,
169     },
170     {
171       id: "wolf",
172       name: "Wolf",
173       avatar: "bear",
174       color: "#475569",
175       x: 520,
176       y: 250,
177     },
178   ],
179   actions: [
180     { type: "appear", characterId: "fox" },
181     { type: "appear", characterId: "wolf" },
182     {
183       type: "speak",
184       characterId: "fox",
185       text: "Better come back another day...",
186       durationMs: 1800,
187     },
188     { type: "move", characterId: "fox", dx: -380, dy: 0, durationMs: 1300 },
189   ],
190 },
191 ],
192 };
```

fim de examples/fox-and-wolf.ts

package.json

Arquivo 7 de 25 · JSON · 52 linhas · 1.567 bytes

SHA-256: 42de962075f9fe347007a4968fa4b2672e33c180e77352dc2895e3388615c66c

```
1  {
2    "name": "fable-engine",
3    "version": "1.0.0",
4    "description": "A tiny, framework-free TypeScript engine for interactive narratives, with a Canvas 2D
player and self-contained HTML export.",
5    "type": "module",
6    "license": "MIT",
7    "author": "Randerson Oliveira Melville Rebouças",
8    "contributors": [
9      "Marcelo Magalhães Foohs (pedagogical design – Active Remediation Cycle)"
10   ],
11   "homepage": "https://github.com/RandMelville/fable-engine#readme",
12   "repository": {
13     "type": "git",
14     "url": "git+https://github.com/RandMelville/fable-engine.git"
15   },
16   "bugs": {
17     "url": "https://github.com/RandMelville/fable-engine/issues"
18   },
19   "keywords": [
20     "interactive-narrative",
21     "narrative-engine",
22     "computational-thinking",
23     "educational-technology",
24     "canvas",
25     "file-over-app"
26   ],
27   "main": "./dist/index.js",
28   "module": "./dist/index.js",
29   "types": "./dist/index.d.ts",
30   "exports": {
31     ".": {
32       "types": "./dist/index.d.ts",
33       "import": "./dist/index.js"
34     }
35   },
36   "files": [
37     "dist"
38   ],
39   "scripts": {
40     "build:standalone": "node scripts/build-standalone.mjs",
41     "build": "npm run build:standalone && esbuild src/index.ts --bundle --format=esm --
outfile=dist/index.js && tsc -p tsconfig.build.json --emitDeclarationOnly",
42     "test": "node scripts/run-selfcheck.mjs",
43     "test:e2e": "node test/e2e.mjs",
44     "test:all": "npm test && npm run test:e2e",
45     "typecheck": "tsc --noEmit"
46   },
47   "devDependencies": {
48     "esbuild": "^0.21.0",
49     "playwright": "^1.61.1",
50     "typescript": "^5.4.0"
51   }
52 }
```

fim de package.json

scripts/build-standalone.mjs

Arquivo 8 de 25 · JavaScript (ESM) · 27 linhas · 917 bytes

SHA-256: 1c9fc04fb9c0de5532fd8bab3bbef0e1398326582048979bc5967b592f5971aa

```
1  // Bundles src/standalone.ts into a minified IIFE string and writes it to
2  // src/generated/standaloneBundle.ts, so export.ts can inline the player runtime
3  // into the self-contained HTML.
4
5  import { build } from "esbuild";
6  import { writeFileSync } from "node:fs";
7  import { fileURLToPath } from "node:url";
8  import { dirname, resolve } from "node:path";
9
10 const root = resolve(dirname(fileURLToPath(import.meta.url)), "..");
11
12 const result = await build({
13   entryPoints: [resolve(root, "src/standalone.ts")],
14   bundle: true,
15   minify: true,
16   format: "iife",
17   target: "es2020",
18   write: false,
19 });
20
21 const code = result.outputFiles[0].text;
22 const out = `// Generated by scripts/build-standalone.mjs - do not edit by hand.
23 export const standaloneBundle = ${JSON.stringify(code)};
24 `;
25
26 writeFileSync(resolve(root, "src/generated/standaloneBundle.ts"), out);
27 console.log(`build:standalone -> ${code.length} bytes bundled`);
```

fim de scripts/build-standalone.mjs

scripts/run-selfcheck.mjs

Arquivo 9 de 25 · JavaScript (ESM) · 27 linhas · 877 bytes

SHA-256: a3be085eb71638caf26b1f4c3731b07924c90efb70abf2d6db88a89b71c3f6e5

```
1  // Bundles test/selfcheck.ts into a temp .mjs and runs it with node.
2  // Keeps the test headless and framework-free (exits non-zero on failure).
3
4  import { build } from "esbuild";
5  import { execFileSync } from "node:child_process";
6  import { mkdtempSync, rmSync } from "node:fs";
7  import { tmpdir } from "node:os";
8  import { fileURLToPath } from "node:url";
9  import { dirname, resolve, join } from "node:path";
10
11 const root = resolve(dirname(fileURLToPath(import.meta.url)), "..");
12 const dir = mkdtempSync(join(tmpdir(), "fable-selfcheck-"));
13 const outfile = join(dir, "selfcheck.mjs");
14
15 try {
16   await build({
17     entryPoints: [resolve(root, "test/selfcheck.ts")],
18     bundle: true,
19     format: "esm",
20     platform: "node",
21     target: "es2020",
22     outfile,
23   });
24   execFileSync("node", [outfile], { stdio: "inherit" });
25 } finally {
26   rmSync(dir, { recursive: true, force: true });
27 }
```

fim de scripts/run-selfcheck.mjs

src/avatars.ts

Arquivo 10 de 25 · TypeScript · 34 linhas · 1.192 bytes

SHA-256: 4b58aefefdef605b35f68fe07efe80cd80f71a2e9aa5892103d33442a46d15cc

```
1 // Curated avatar palette. Each avatar = a drawn KIND (silhouette by
2 // ears/antenna/beak, see drawCharacter.ts) plus a COLOR. A small, closed set:
3 // the author PICKS one when creating a character – there is no free graphic
4 // authoring.
5
6 import type { AvatarKind } from "../drawCharacter";
7
8 export interface Avatar {
9   kind: AvatarKind;
10  color: string;
11 }
12
13 export const AVATARS: Avatar[] = [
14   { kind: "round", color: "#7C3AED" },
15   { kind: "cat", color: "#E11D48" },
16   { kind: "rabbit", color: "#D97706" },
17   { kind: "bear", color: "#DB2777" },
18   { kind: "bird", color: "#2563EB" },
19   { kind: "robot", color: "#0891B2" },
20   { kind: "round", color: "#059669" },
21   { kind: "cat", color: "#CA8A04" },
22   { kind: "rabbit", color: "#9333EA" },
23   { kind: "bear", color: "#65A30D" },
24   { kind: "bird", color: "#0D9488" },
25   { kind: "robot", color: "#475569" },
26 ];
27
28 /** Default avatar (first of the palette) for a new character. */
29 export const DEFAULT_AVATAR: Avatar = AVATARS[0];
30
31 /** Avatar equality (kind + color) – used to highlight the selection in a picker. */
32 export function sameAvatar(a: Avatar, kind: string, color: string): boolean {
33   return a.kind === kind && a.color === color;
34 }
```

fim de src/avatars.ts

src/backgrounds.ts

Arquivo 11 de 25 · TypeScript · 26 linhas · 1.110 bytes

SHA-256: fe5d6ba8679baf12b82b7df1a21ba9aa39bf5ffbe34399d389f02ca928104e1e

```
1  // Curated backgrounds per scene. A small, closed set: the author PICKS a scene's
2  // background from a menu – they do not draw it. Each background is defined by
3  // colors (a sky gradient plus the ground), so the player can render it without
4  // images (keeps the "file over app" export light).
5
6  export interface Background {
7    key: string;
8    skyTop: string;
9    skyBottom: string;
10   ground: string;
11 }
12
13 export const BACKGROUNDS: Background[] = [
14   { key: "field", skyTop: "#dbeafe", skyBottom: "#ecfccb", ground: "#86efac" },
15   { key: "forest", skyTop: "#a7f3d0", skyBottom: "#d9f99d", ground: "#22c55e" },
16   { key: "night", skyTop: "#1e293b", skyBottom: "#475569", ground: "#334155" },
17   { key: "beach", skyTop: "#bae6fd", skyBottom: "#e0f2fe", ground: "#fde68a" },
18   { key: "city", skyTop: "#e2e8f0", skyBottom: "#f1f5f9", ground: "#94a3b8" },
19 ];
20
21 const DEFAULT_BACKGROUND = BACKGROUNDS[0];
22
23 /** Resolves the background key (empty/unknown → default "field"). */
24 export function resolveBackground(key: string | undefined): Background {
25   return BACKGROUNDS.find((b) => b.key === key) ?? DEFAULT_BACKGROUND;
26 }
```

fim de src/backgrounds.ts

src/drawBackground.ts

Arquivo 12 de 25 · TypeScript · 189 linhas · 4.924 bytes

SHA-256: 9aac88a87d8a50aabe110f1aa0e82a80924f00cf99573d310b99218969d2624a

```

1  // Procedural drawing of the curated backgrounds (Canvas 2D, framework-free, no
2  // external images – keeps the "file over app" export light and offline).
3  //
4  // Each background paints simple vector elements (trees, sun, moon, stars,
5  // buildings, waves) over the sky/ground already filled by the player. Fixed
6  // positions (no randomness) for a stable, reproducible look.
7
8  const GROUND_Y = 300;
9
10 // --- Primitives -----
11
12 function leafyTree(ctx: CanvasRenderingContext2D, x: number, scale = 1): void {
13   const base = GROUND_Y + 6;
14   ctx.fillStyle = "#92400e";
15   ctx.fillRect(x - 5 * scale, base - 34 * scale, 10 * scale, 34 * scale);
16   ctx.fillStyle = "#16a34a";
17   for (const [dy, r] of [
18     [54, 30],
19     [74, 26],
20     [92, 20],
21   ] as const) {
22     ctx.beginPath();
23     ctx.arc(x, base - dy * scale, r * scale, 0, Math.PI * 2);
24     ctx.fill();
25   }
26 }
27
28 function pineTree(
29   ctx: CanvasRenderingContext2D,
30   x: number,
31   scale = 1,
32   color = "#15803d",
33 ): void {
34   const base = GROUND_Y + 6;
35   ctx.fillStyle = "#92400e";
36   ctx.fillRect(x - 4 * scale, base - 16 * scale, 8 * scale, 16 * scale);
37   ctx.fillStyle = color;
38   for (const [dy, width, height] of [
39     [16, 34, 34],
40     [38, 28, 30],
41     [58, 22, 26],
42   ] as const) {
43     ctx.beginPath();
44     ctx.moveTo(x, base - (dy + height) * scale);
45     ctx.lineTo(x - width * scale, base - dy * scale);
46     ctx.lineTo(x + width * scale, base - dy * scale);
47     ctx.closePath();
48     ctx.fill();
49   }
50 }
51
52 function cloud(ctx: CanvasRenderingContext2D, x: number, y: number): void {
53   ctx.fillStyle = "rgba(255,255,255,0.9)";
54   for (const [dx, dy, r] of [
55     [0, 0, 18],
56     [22, 4, 14],
57     [-20, 6, 13],
58     [6, -8, 14],
59   ] as const) {
60     ctx.beginPath();
61     ctx.arc(x + dx, y + dy, r, 0, Math.PI * 2);
62     ctx.fill();
63   }
64 }
65
66 function sun(ctx: CanvasRenderingContext2D, x: number, y: number): void {
67   ctx.strokeStyle = "rgba(250,204,21,0.8)";

```

```
68   ctx.lineWidth = 3;
69   for (let i = 0; i < 8; i++) {
70     const a = (i / 8) * Math.PI * 2;
71     ctx.beginPath();
72     ctx.moveTo(x + Math.cos(a) * 26, y + Math.sin(a) * 26);
73     ctx.lineTo(x + Math.cos(a) * 36, y + Math.sin(a) * 36);
74     ctx.stroke();
75   }
76   ctx.fillStyle = "#fde047";
77   ctx.beginPath();
78   ctx.arc(x, y, 22, 0, Math.PI * 2);
79   ctx.fill();
80 }
81
82 function moon(ctx: CanvasRenderingContext2D, x: number, y: number): void {
83   ctx.fillStyle = "#e5e7eb";
84   ctx.beginPath();
85   ctx.arc(x, y, 20, 0, Math.PI * 2);
86   ctx.fill();
87 }
88
89 function stars(ctx: CanvasRenderingContext2D): void {
90   ctx.fillStyle = "#fde68a";
91   const pts = [
92     [60, 50],
93     [140, 90],
94     [220, 40],
95     [300, 80],
96     [420, 55],
97     [520, 95],
98     [600, 45],
99     [660, 100],
100    [180, 150],
101    [560, 150],
102  ] as const;
103   for (const [x, y] of pts) {
104     ctx.beginPath();
105     ctx.arc(x, y, 2.2, 0, Math.PI * 2);
106     ctx.fill();
107   }
108 }
109
110 function building(
111   ctx: CanvasRenderingContext2D,
112   x: number,
113   width: number,
114   height: number,
115 ): void {
116   const top = GROUND_Y - height;
117   ctx.fillStyle = "#64748b";
118   ctx.fillRect(x, top, width, height);
119   ctx.fillStyle = "#fde047";
120   for (let wy = top + 10; wy < GROUND_Y - 10; wy += 22) {
121     for (let wx = x + 8; wx < x + width - 8; wx += 20) {
122       ctx.fillRect(wx, wy, 8, 10);
123     }
124   }
125 }
126
127 function waves(ctx: CanvasRenderingContext2D): void {
128   ctx.strokeStyle = "rgba(56,189,248,0.7)";
129   ctx.lineWidth = 3;
130   for (let row = 0; row < 3; row++) {
131     const y = GROUND_Y + 16 + row * 20;
132     ctx.beginPath();
133     for (let x = 0; x <= 720; x += 40) {
134       ctx.moveTo(x, y);
135       ctx.quadraticCurveTo(x + 10, y - 6, x + 20, y);
136       ctx.quadraticCurveTo(x + 30, y + 6, x + 40, y);
137     }
138     ctx.stroke();
139   }
```

```
140 | }
141 |
142 | // --- Composition per background -----
143 |
144 | export function drawBackground(
145 |   ctx: CanvasRenderingContext2D,
146 |   key: string | undefined,
147 | ): void {
148 |   switch (key) {
149 |     case "forest":
150 |       cloud(ctx, 580, 70);
151 |       pineTree(ctx, 80, 1.1);
152 |       pineTree(ctx, 180, 0.9);
153 |       pineTree(ctx, 300, 1.2);
154 |       pineTree(ctx, 470, 1);
155 |       pineTree(ctx, 600, 1.15);
156 |       pineTree(ctx, 680, 0.85);
157 |       break;
158 |     case "night":
159 |       stars(ctx);
160 |       moon(ctx, 620, 70);
161 |       pineTree(ctx, 110, 1, "#14532d");
162 |       pineTree(ctx, 250, 1.1, "#14532d");
163 |       pineTree(ctx, 520, 0.95, "#14532d");
164 |       break;
165 |     case "beach":
166 |       sun(ctx, 110, 80);
167 |       cloud(ctx, 470, 70);
168 |       waves(ctx);
169 |       break;
170 |     case "city":
171 |       cloud(ctx, 150, 70);
172 |       building(ctx, 70, 70, 150);
173 |       building(ctx, 170, 90, 210);
174 |       building(ctx, 290, 60, 120);
175 |       building(ctx, 380, 100, 190);
176 |       building(ctx, 510, 80, 240);
177 |       building(ctx, 620, 70, 150);
178 |       break;
179 |     case "field":
180 |     default:
181 |       sun(ctx, 90, 70);
182 |       cloud(ctx, 330, 60);
183 |       cloud(ctx, 560, 90);
184 |       leafyTree(ctx, 150, 0.9);
185 |       leafyTree(ctx, 520, 1.05);
186 |       leafyTree(ctx, 650, 0.8);
187 |       break;
188 |   }
189 | }
```

fim de src/drawBackground.ts

src/drawCharacter.ts

Arquivo 13 de 25 · TypeScript · 240 linhas · 6.632 bytes

SHA-256: 808a644a35ec383c207e3d23396128a918102ce7cc6d6c3bd387c47901451455

```

1  // Draws the CHARACTER on the Canvas (framework-free, no images).
2  //
3  // The character is a mascot DRAWN with primitives (like the background in
4  // drawBackground.ts), whose FACE changes expression according to the mood – no
5  // speech bubble emotion, no emoji. A curated set of "kinds" (silhouette by
6  // ears/antenna/beak) plus a color. The same function runs in the player and in
7  // the "file over app" export (pure Canvas 2D).
8
9  export type AvatarKind = "round" | "cat" | "rabbit" | "bear" | "bird" | "robot";
10
11 export interface AvatarKindDef {
12   key: AvatarKind;
13   name: string;
14 }
15
16 export const AVATAR_KINDS: AvatarKindDef[] = [
17   { key: "round", name: "Round" },
18   { key: "cat", name: "Cat" },
19   { key: "rabbit", name: "Rabbit" },
20   { key: "bear", name: "Bear" },
21   { key: "bird", name: "Bird" },
22   { key: "robot", name: "Robot" },
23 ];
24
25 export interface ExpressionState {
26   mood?: string;
27   speaking?: boolean;
28 }
29
30 const EYE_X = 9;
31 const EYE_Y = -6;
32 const DARK = "#1f2937";
33
34 /** Darkens a hex color (#rrggbb) by a factor (0..1) – body outline/shadow. */
35 function darken(hex: string, factor = 0.78): string {
36   const m = /^#?([0-9a-f]{6})$/i.exec(hex.trim());
37   if (!m) return "#4b5563";
38   const n = parseInt(m[1], 16);
39   const r = Math.round(((n >> 16) & 255) * factor);
40   const g = Math.round(((n >> 8) & 255) * factor);
41   const b = Math.round((n & 255) * factor);
42   return `rgb(${r},${g},${b})`;
43 }
44
45 /** Ears/antenna/beak of the kind, drawn BEFORE the body (they sit behind). */
46 function drawFeatureBehind(
47   ctx: CanvasRenderingContext2D,
48   kind: string,
49   color: string,
50 ): void {
51   const dark = darken(color);
52   ctx.fillStyle = color;
53   ctx.strokeStyle = dark;
54   ctx.lineWidth = 2;
55   if (kind === "cat") {
56     for (const sx of [-1, 1]) {
57       ctx.beginPath();
58       ctx.moveTo(sx * 9, -26);
59       ctx.lineTo(sx * 20, -40);
60       ctx.lineTo(sx * 22, -22);
61       ctx.closePath();
62       ctx.fill();
63     }
64   } else if (kind === "rabbit") {
65     for (const sx of [-1, 1]) {
66       ctx.beginPath();
67       ctx.ellipse(sx * 9, -42, 6, 18, sx * 0.2, 0, Math.PI * 2);

```

```
68     ctx.fill();
69   }
70   } else if (kind === "bear") {
71     for (const sx of [-1, 1]) {
72       ctx.beginPath();
73       ctx.arc(sx * 16, -26, 9, 0, Math.PI * 2);
74       ctx.fill();
75     }
76   } else if (kind === "robot") {
77     ctx.beginPath();
78     ctx.moveTo(0, -28);
79     ctx.lineTo(0, -42);
80     ctx.stroke();
81     ctx.beginPath();
82     ctx.arc(0, -45, 4, 0, Math.PI * 2);
83     ctx.fill();
84   } else if (kind === "bird") {
85     for (const sx of [-1, 1]) {
86       ctx.beginPath();
87       ctx.ellipse(sx * 26, 4, 8, 14, sx * 0.5, 0, Math.PI * 2);
88       ctx.fill();
89     }
90   }
91 }
92
93 /** Eyes according to the mood (happy arcs or spheres with a pupil). */
94 function drawEyes(ctx: CanvasRenderingContext2D, mood: string): void {
95   const happyArc = mood === "happy" || mood === "loving";
96   const big = mood === "surprised" || mood === "scared";
97   if (happyArc) {
98     ctx.strokeStyle = DARK;
99     ctx.lineWidth = 2.5;
100    for (const sx of [-1, 1]) {
101      ctx.beginPath();
102      ctx.arc(sx * EYE_X, EYE_Y + 1, 5, Math.PI * 1.15, Math.PI * 1.85);
103      ctx.stroke();
104    }
105    return;
106  }
107  const r = big ? 6.5 : 5;
108  const pupil = big ? 3.2 : 2.6;
109  // look up when "thinking"
110  const dy = mood === "thinking" ? -1.5 : 0;
111  for (const sx of [-1, 1]) {
112    ctx.fillStyle = "#ffffff";
113    ctx.beginPath();
114    ctx.arc(sx * EYE_X, EYE_Y, r, 0, Math.PI * 2);
115    ctx.fill();
116    ctx.fillStyle = DARK;
117    ctx.beginPath();
118    ctx.arc(sx * EYE_X, EYE_Y + dy, pupil, 0, Math.PI * 2);
119    ctx.fill();
120  }
121  // angry eyebrows
122  if (mood === "angry") {
123    ctx.strokeStyle = DARK;
124    ctx.lineWidth = 2.5;
125    for (const sx of [-1, 1]) {
126      ctx.beginPath();
127      ctx.moveTo(sx * (EYE_X + 5), EYE_Y - 9);
128      ctx.lineTo(sx * (EYE_X - 3), EYE_Y - 5);
129      ctx.stroke();
130    }
131  }
132 }
133
134 /** Mouth according to the mood (smile, frown, "O", open...). */
135 function drawMouth(
136   ctx: CanvasRenderingContext2D,
137   mood: string,
138   speaking: boolean,
139 ): void {
```

```
140   ctx.strokeStyle = DARK;
141   ctx.fillStyle = DARK;
142   ctx.lineWidth = 2.5;
143   const y = 9;
144   if (mood === "happy" || mood === "loving") {
145     ctx.beginPath();
146     ctx.arc(0, y - 2, 7, Math.PI * 0.15, Math.PI * 0.85);
147     ctx.stroke();
148   } else if (mood === "sad") {
149     ctx.beginPath();
150     ctx.arc(0, y + 7, 7, Math.PI * 1.2, Math.PI * 1.8);
151     ctx.stroke();
152   } else if (mood === "surprised") {
153     ctx.beginPath();
154     ctx.arc(0, y + 1, 3.5, 0, Math.PI * 2);
155     ctx.stroke();
156   } else if (mood === "scared") {
157     ctx.beginPath();
158     ctx.ellipse(0, y + 1, 4, 5.5, 0, 0, Math.PI * 2);
159     ctx.fill();
160   } else if (mood === "thinking") {
161     ctx.beginPath();
162     ctx.moveTo(-2, y);
163     ctx.lineTo(5, y);
164     ctx.stroke();
165   } else if (mood === "angry") {
166     ctx.beginPath();
167     ctx.arc(0, y + 6, 6, Math.PI * 1.25, Math.PI * 1.75);
168     ctx.stroke();
169   } else if (speaking) {
170     ctx.beginPath();
171     ctx.ellipse(0, y, 3.5, 3, 0, 0, Math.PI * 2);
172     ctx.fill();
173   } else {
174     // neutral
175     ctx.beginPath();
176     ctx.moveTo(-4, y);
177     ctx.lineTo(4, y);
178     ctx.stroke();
179   }
180 }
181
182 /**
183  * Draws the character centered at ``(x, y)``. ``mood`` sets the facial
184  * expression; ``speaking`` opens the mouth slightly when there is no specific mood.
185  */
186 export function drawCharacter(
187   ctx: CanvasRenderingContext2D,
188   opts: { avatar: string; color: string; x: number; y: number; scale?: number },
189   state: ExpressionState = {},
190 ): void {
191   const { avatar, color, x, y, scale = 1 } = opts;
192   const mood = state.mood ?? "";
193   ctx.save();
194   ctx.translate(x, y);
195   ctx.scale(scale, scale);
196
197   drawFeatureBehind(ctx, avatar, color);
198
199   // Body (ellipse) with a darker outline.
200   ctx.fillStyle = color;
201   ctx.strokeStyle = darken(color);
202   ctx.lineWidth = 2;
203   ctx.beginPath();
204   ctx.ellipse(0, 0, 26, 30, 0, 0, Math.PI * 2);
205   ctx.fill();
206   ctx.stroke();
207
208   // Light belly (charm).
209   ctx.fillStyle = "rgba(255,255,255,0.18)";
210   ctx.beginPath();
211   ctx.ellipse(0, 8, 15, 18, 0, 0, Math.PI * 2);
```

```
212     ctx.fill();
213
214     // Bird beak (in front of the body, below the eyes).
215     if (avatar === "bird") {
216         ctx.fillStyle = "#f59e0b";
217         ctx.beginPath();
218         ctx.moveTo(-5, 2);
219         ctx.lineTo(5, 2);
220         ctx.lineTo(0, 9);
221         ctx.closePath();
222         ctx.fill();
223     }
224
225     // Rosy cheeks for the loving mood.
226     if (mood === "loving") {
227         ctx.fillStyle = "rgba(244,114,182,0.45)";
228         for (const sx of [-1, 1]) {
229             ctx.beginPath();
230             ctx.arc(sx * 15, 3, 4, 0, Math.PI * 2);
231             ctx.fill();
232         }
233     }
234
235     drawEyes(ctx, mood);
236     // The bird speaks through its beak; avoid a double mouth.
237     if (avatar !== "bird") drawMouth(ctx, mood, !!state.speaking);
238
239     ctx.restore();
240 }
```

fim de src/drawCharacter.ts

src/engine.ts

Arquivo 14 de 25 · TypeScript · 224 linhas · 6.282 bytes

SHA-256: 68b2c5813841e80afaf4de59f91c6fddd0e28cd9376bdc9bdbdc24eb7fd754a7

```
1  /**
2   * Data model and core logic of the story engine.
3   *
4   * Framework-free core: the data-model types, the critical logic (conditional
5   * evaluation, modifiers, structural summary) and the "flattening" of a scene's
6   * animation program. No external dependencies – this is what makes the
7   * self-contained "file over app" export possible.
8   */
9
10 // -----
11 // Data model
12 // -----
13
14 export type Operation = "+" | "-" | "=";
15 export type Operator = ">" | "<" | ">=" | "<=" | "==" | "!=";
16 export type Value = number | boolean;
17
18 export interface Variable {
19   id: string;
20   name: string;
21   type: "number" | "boolean";
22   initialValue: Value;
23 }
24
25 export interface Character {
26   id: string;
27   name: string;
28   avatar: string; // kind of avatar drawn (see drawCharacter.ts)
29   color: string;
30   x: number; // initial position in this scene (px on the stage)
31   y: number;
32   /** "Visual reacts to a variable": when the condition holds, use this avatar. */
33   rule?: {
34     variableId: string;
35     operator: Operator;
36     value: Value;
37     avatar: string;
38     color: string;
39   };
40 }
41
42 /** A scene's animation program. `repeat` replays visual movement; it is not counted as an authored loop.
43 */
44 export type Action =
45   | { type: "appear"; characterId: string }
46   | {
47     type: "speak";
48     characterId: string;
49     text: string;
50     durationMs: number;
51     mood?: string;
52   }
53   | {
54     type: "move";
55     characterId: string;
56     dx: number;
57     dy: number;
58     durationMs: number;
59   }
60   | { type: "wait"; durationMs: number }
61   | { type: "repeat"; times: number; actions: Action[] };
62
63 export interface Modifier {
64   variableId: string;
65   operation: Operation;
66   value: Value;
67 }
```

```
67
68 export interface Choice {
69   id: string;
70   buttonLabel: string;
71   targetSceneId: string;
72   modifiers: Modifier[];
73 }
74
75 export interface Conditional {
76   variableId: string;
77   operator: Operator;
78   value: Value;
79   sceneIfTrueId: string;
80   sceneIfFalseId: string;
81 }
82
83 export interface Scene {
84   id: string;
85   title: string;
86   text?: string;
87   /** Key of the curated background (resolved to colors in the player). */
88   background?: string;
89   characters: Character[];
90   actions: Action[];
91   /** If present, the scene waits for the reader's choice (interactive branch). */
92   choices?: Choice[];
93   /** If present (and no choices), branches by state at the end of the animation. */
94   conditional?: Conditional;
95 }
96
97 export interface Story {
98   formatVersion: 1;
99   title: string;
100   author: string;
101   variables: Variable[];
102   scenes: Scene[];
103   initialSceneId: string;
104 }
105
106 export type State = Record<string, Value>;
107
108 // -----
109 // Core logic
110 // -----
111
112 /** Initial narrative state derived from the declared variables. */
113 export function initialState(story: Story): State {
114   const state: State = {};
115   for (const v of story.variables) state[v.id] = v.initialValue;
116   return state;
117 }
118
119 /** Evaluates a conditional against the current value of a variable. */
120 export function evaluateConditional(
121   current: Value,
122   operator: Operator,
123   target: Value,
124 ): boolean {
125   switch (operator) {
126     case "==":
127       return current === target;
128     case "!=":
129       return current !== target;
130     case ">":
131       return Number(current) > Number(target);
132     case "<":
133       return Number(current) < Number(target);
134     case ">=":
135       return Number(current) >= Number(target);
136     case "<=":
137       return Number(current) <= Number(target);
138   }
```

```

139   }
140
141   /** Applies a choice modifier to the state (effect on one variable). */
142   export function applyModifier(state: State, mod: Modifier): State {
143     const next: State = { ...state };
144     const current = next[mod.variableId];
145     if (mod.operation === "=") {
146       next[mod.variableId] = mod.value;
147     } else if (mod.operation === "+") {
148       next[mod.variableId] = Number(current) + Number(mod.value);
149     } else {
150       next[mod.variableId] = Number(current) - Number(mod.value);
151     }
152     return next;
153   }
154
155   /** Resolves the next scene after a conditional scene, given the state. */
156   export function nextSceneByConditional(cond: Conditional, state: State): string {
157     const ok = evaluateConditional(state[cond.variableId], cond.operator, cond.value);
158     return ok ? cond.sceneIfTrueId : cond.sceneIfFalseId;
159   }
160
161   /**
162    * Summarizes the structural constructs the author built into a story:
163    * scenes (how the story is decomposed), variables (state), branches
164    * (choices + conditionals) and modifiers (state effects). It describes what is
165    * implemented in the narrative – a deterministic account of computational
166    * constructs, not a score of the author's cognitive faculties.
167    */
168   export function summarizeAlgorithmicStructure(story: Story): {
169     scenes: number;
170     variables: number;
171     branches: number;
172     modifiers: number;
173   } {
174     let branches = 0;
175     let modifiers = 0;
176     for (const scene of story.scenes) {
177       if (scene.conditional) branches += 1;
178       for (const choice of scene.choices ?? []) {
179         branches += 1;
180         modifiers += choice.modifiers.length;
181       }
182     }
183     return {
184       scenes: story.scenes.length,
185       variables: story.variables.length,
186       branches,
187       modifiers,
188     };
189   }
190
191   // -----
192   // Animation program: flattening (expands `repeat` loops)
193   // -----
194
195   export type PrimitiveStep =
196     | { type: "appear"; characterId: string }
197     | {
198       type: "speak";
199       characterId: string;
200       text: string;
201       durationMs: number;
202       mood?: string;
203     }
204     | {
205       type: "move";
206       characterId: string;
207       dx: number;
208       dy: number;
209       durationMs: number;
210     }

```

```
211 | { type: "wait"; durationMs: number };
212
213 /** Flattens the scene program, expanding `repeat` into linear steps. */
214 export function flatten(actions: Action[]): PrimitiveStep[] {
215   const out: PrimitiveStep[] = [];
216   for (const a of actions) {
217     if (a.type === "repeat") {
218       for (let i = 0; i < a.times; i++) out.push(...flatten(a.actions));
219     } else {
220       out.push(a);
221     }
222   }
223   return out;
224 }
```

fim de src/engine.ts

src/export.ts

Arquivo 15 de 25 · TypeScript · 89 linhas · 3.498 bytes

SHA-256: c67d3e7cbb3821c08d1874fe26b6c0f1a184f096d3359811ace0d53ff599bb90

```

1 // "File over app" export: the reader takes the story as a self-contained .html
2 // that opens offline in any browser, with no platform.
3 //
4 // The HTML embeds the `Story` (JSON), the player runtime (an IIFE bundle
5 // pre-generated by esbuild) and the stage CSS. ZERO network requests: no
6 // external fonts/CDN (falls back to system-ui).
7
8 import type { Story } from "./engine";
9 import { standaloneBundle } from "./generated/standaloneBundle";
10
11 export interface ExportLabels {
12   end: string;
13   playAgain: string;
14   structure: string;
15 }
16
17 const DEFAULT_LABELS: ExportLabels = {
18   end: "The End 🌿",
19   playAgain: "🔄 Play again",
20   structure: "Structure:",
21 };
22
23 /** Escapes ``</script>`` so inline JSON/JS does not close the tag. */
24 function safe(json: string): string {
25   return json.replace(/<\/(script)/gi, "<\\/$1");
26 }
27
28 /** Builds the self-contained HTML for a story (a string ready to save). */
29 export function exportHtml(
30   story: Story,
31   options: { labels?: Partial<ExportLabels> } = {},
32 ): string {
33   const labels: ExportLabels = { ...DEFAULT_LABELS, ...options.labels };
34   const title = (story.title || "My story").replace(/[<>]/g, "");
35   const author = (story.author || "").replace(/[<>]/g, "");
36   return `<!doctype html>
37 <html lang="en">
38 <head>
39 <meta charset="utf-8" />
40 <meta name="viewport" content="width=device-width, initial-scale=1" />
41 <title>${title}</title>
42 <style>
43   :root { font-family: system-ui, -apple-system, sans-serif; }
44   body { margin: 0; background: #f4f4f5; color: #27272a; display: flex; min-height: 100vh;
45     align-items: center; justify-content: center; padding: 16px; box-sizing: border-box; }
46   .wrap { width: 100%; max-width: 760px; }
47   h1 { font-size: 18px; margin: 0 0 4px; }
48   .meta { color: #71717a; font-size: 13px; margin: 0 0 12px; }
49   .stage-wrap { background: #fff; border-radius: 16px; padding: 16px;
50     box-shadow: 0 6px 24px rgba(0,0,0,0.08); }
51   .stage { width: 100%; height: auto; border-radius: 12px; display: block; background: #dbeafe; }
52   .choices { display: flex; flex-wrap: wrap; gap: 10px; margin-top: 14px; align-items: center; }
53   .choice-btn { background: #fff; color: #5b21b6; border: 2px solid #7c3aed;
54     border-radius: 12px; padding: 10px 16px; font: inherit; font-weight: 600; cursor: pointer; }
55   .choice-btn:hover { background: #f5f3ff; }
56   .choice-btn.secondary { border-color: #a1a1aa; color: #52525b; }
57   .ending { font-size: 18px; font-weight: 700; color: #5b21b6; }
58   .structure { margin-top: 14px; font: 600 12px ui-monospace, monospace; color: #10b981;
59     background: #ecfdf5; border-radius: 10px; padding: 8px 12px; }
60   footer { margin-top: 12px; text-align: center; color: #a1a1aa; font-size: 11px; }
61 </style>
62 </head>
63 <body>
64 <div class="wrap">
65   <h1>${title}</h1>
66   ${author ? `<p class="meta">by ${author}</p>` : ""}
67   <div class="stage-wrap"><div id="stage"></div></div>

```

```
68 |     <footer>Interactive story.</footer>
69 | </div>
70 | <script>window.__STORY__=${safe(JSON.stringify(story))};window.__LABELS__=${safe(
71 |     JSON.stringify(labels),
72 |     )};</script>
73 | <script>${standaloneBundle}</script>
74 | </body>
75 | </html>`;
76 | }
77 |
78 | /** Triggers the download of the HTML as a file (in the browser). */
79 | export function downloadHtml(filename: string, html: string): void {
80 |     const blob = new Blob([html], { type: "text/html;charset=utf-8" });
81 |     const url = URL.createObjectURL(blob);
82 |     const a = document.createElement("a");
83 |     a.href = url;
84 |     a.download = filename;
85 |     document.body.appendChild(a);
86 |     a.click();
87 |     a.remove();
88 |     URL.revokeObjectURL(url);
89 | }
```

fim de src/export.ts

src/generated/standaloneBundle.ts

Arquivo 16 de 25 · TypeScript · 2 linhas · 13.007 bytes

SHA-256: 5ff05c9bc2d3dcb4f783a2cff026f3abaa993563a368b876b05956dd47fb9d15

Artefato gerado pelo build (bundle do player embutido no HTML exportado); derivado de src/standalone.ts.

```
1 | // Generated by scripts/build-standalone.mjs – do not edit by hand.
```

```

2 export const standaloneBundle = "\"use strict\";(()=>{function k(e){let i={};for(let t of
e.variables)i[t.id]=t.initialValue;return i}function M(e,i,t){switch(i){case\"==\":return
e===t;case\"!=\":return e!==t;case\">\":return Number(e)>Number(t);case\"<\":return Number(e)
<Number(t);case\">=\":return Number(e)>=Number(t);case\"<=\":return Number(e)<=Number(t)}}function C(e,i)
{let t={...e},n=t[i.variableId];return i.operation===\"=\"?t[i.variableId]=i.value:i.operation===\"+\"?
t[i.variableId]=Number(n)+Number(i.value):t[i.variableId]=Number(n)-Number(i.value),t}function E(e,i)
{return M(i[e.variableId],e.operator,e.value)?e.sceneIfTrueId:e.sceneIfFalseId}function R(e){let
i=0,t=0;for(let n of e.scenes){n.conditional&&(i+=1);for(let s of n.choices??
[])i+=1,t+=s.modifiers.length}return{scenes:e.scenes.length,variables:e.variables.length,branches:i,modif
iers:t}}function I(e){let i=[];for(let t of e)if(t.type===\"repeat\")for(let
n=0;n<t.times;n++)i.push(...I(t.actions));else i.push(t);return i}var w=
[{key:\"field\",skyTop:\"#dbeafe\",skyBottom:\"#ecfccb\",ground:\"#86efac\"},
{key:\"forest\",skyTop:\"#a7f3d0\",skyBottom:\"#d9f99d\",ground:\"#22c55e\"},
{key:\"night\",skyTop:\"#1e293b\",skyBottom:\"#475569\",ground:\"#334155\"},
{key:\"beach\",skyTop:\"#bae6fd\",skyBottom:\"#e0f2fe\",ground:\"#fde68a\"},
{key:\"city\",skyTop:\"#e2e8f0\",skyBottom:\"#f1f5f9\",ground:\"#94a3b8\"}],v=w[0];function B(e){return
w.find(i=>i.key===e)?v}function T(e,i,t=1){e.fillStyle=\"#92400e\",e.fillRect(i-5*t,306-
34*t,10*t,34*t),e.fillStyle=\"#16a34a\";for(let[s,r]of[[54,30],[74,26],
[92,20]])e.beginPath(),e.arc(i,306-s*t,r*t,0,Math.PI*2),e.fill()}function f(e,i,t=1,n=\"#15803d\")
{e.fillStyle=\"#92400e\",e.fillRect(i-4*t,306-16*t,8*t,16*t),e.fillStyle=n;for(let[r,a,o]of[[16,34,34],
[38,28,30],[58,22,26]])e.beginPath(),e.moveTo(i,306-(r+o)*t),e.lineTo(i-a*t,306-r*t),e.lineTo(i+a*t,306-
r*t),e.closePath(),e.fill()}function p(e,i,t)
{e.fillStyle=\"rgba(255,255,255,0.9)\";for(let[n,s,r]of[[0,0,18],[22,4,14],[-20,6,13],
[6,-8,14]])e.beginPath(),e.arc(i+n,t+s,r,0,Math.PI*2),e.fill()}function D(e,i,t)
{e.strokeStyle=\"rgba(250,204,21,0.8)\";e.lineWidth=3;for(let n=0;n<8;n++){let
s=n/8*Math.PI*2;e.beginPath(),e.moveTo(i+Math.cos(s)*26,t+Math.sin(s)*26),e.lineTo(i+Math.cos(s)*36,t+Mat
h.sin(s)*36),e.stroke()}e.fillStyle=\"#fde047\",e.beginPath(),e.arc(i,t,22,0,Math.PI*2),e.fill()}function
H(e,i,t){e.fillStyle=\"#e5e7eb\",e.beginPath(),e.arc(i,t,20,0,Math.PI*2),e.fill()}function $(e)
{e.fillStyle=\"#fde68a\";let i=[[60,50],[140,90],[220,40],[300,80],[420,55],[520,95],[600,45],[660,100],
[180,150],[560,150]];for(let[t,n]of i)e.beginPath(),e.arc(t,n,2.2,0,Math.PI*2),e.fill()}function
u(e,i,t,n){let s=300-n;e.fillStyle=\"#64748b\",e.fillRect(i,s,t,n),e.fillStyle=\"#fde047\";for(let
r=s+10;r<290;r+=22)for(let a=i+8;a<i+t-8;a+=20)e.fillRect(a,r,8,10)}function U(e)
{e.strokeStyle=\"rgba(56,189,248,0.7)\";e.lineWidth=3;for(let i=0;i<3;i++){let
t=316+i*20;e.beginPath();for(let n=0;n<=720;n+=40)e.moveTo(n,t),e.quadraticCurveTo(n+10,t-
6,n+20,t),e.quadraticCurveTo(n+30,t+6,n+40,t);e.stroke()}function A(e,i){switch(i)
{case\"forest\":p(e,580,70),f(e,80,1.1),f(e,180,.9),f(e,300,1.2),f(e,470,1),f(e,600,1.15),f(e,680,.85);br
eak;case\"night\":$(e),H(e,620,70),f(e,110,1,\"#14532d\"),f(e,250,1.1,\"#14532d\"),f(e,520,.95,\"#14532d\
\");break;case\"beach\":D(e,110,80),p(e,470,70),U(e);break;case\"city\":p(e,150,70),u(e,70,70,150),u(e,170
,90,210),u(e,290,60,120),u(e,380,100,190),u(e,510,80,240),u(e,620,70,150);break;case\"field\":default:D(e
,90,70),p(e,330,60),p(e,560,90),T(e,150,.9),T(e,520,1.05),T(e,650,.8);break}}var
c=9,b=-6,g=\"#1f2937\";function N(e,i=.78){let t=/^#?([0-9a-f]
{6})$/i.exec(e.trim());if(!t)return\"#4b5563\";let
n=parseInt(t[1],16),s=Math.round((n>>16&255)*i),r=Math.round((n>>8&255)*i),a=Math.round((n&255)*i);return
`rgb(${s},${r},${a})`}function Y(e,i,t){let
n=N(t);if(e.fillStyle=t,e.strokeStyle=n,e.lineWidth=2,i===\"cat\")for(let s
of[-1,1])e.beginPath(),e.moveTo(s*9,-26),e.lineTo(s*20,-40),e.lineTo(s*22,-22),e.closePath(),e.fill();els
e if(i===\"rabbit\")for(let s
of[-1,1])e.beginPath(),e.ellipse(s*9,-42,6,18,s*.2,0,Math.PI*2),e.fill();else if(i===\"bear\")for(let s
of[-1,1])e.beginPath(),e.arc(s*16,-26,9,0,Math.PI*2),e.fill();else
if(i===\"robot\")e.beginPath(),e.moveTo(0,-28),e.lineTo(0,-42),e.stroke(),e.beginPath(),e.arc(0,-45,4,0,M
ath.PI*2),e.fill();else if(i===\"bird\")for(let s
of[-1,1])e.beginPath(),e.ellipse(s*26,4,8,14,s*.5,0,Math.PI*2),e.fill()}function G(e,i){let
t=i===\"happy\"||i===\"loving\",n=i===\"surprised\"||i===\"scared\";if(t)
{e.strokeStyle=g,e.lineWidth=2.5;for(let o
of[-1,1])e.beginPath(),e.arc(o*c,b+1,5,Math.PI*1.15,Math.PI*1.85),e.stroke();return}let s=n?6.5:5,r=n?
3.2:2.6,a=i===\"thinking\"?-1.5:0;for(let o
of[-1,1])e.fillStyle=\"#ffffff\",e.beginPath(),e.arc(o*c,b,s,0,Math.PI*2),e.fill(),e.fillStyle=g,e.beginP
ath(),e.arc(o*c,b+a,r,0,Math.PI*2),e.fill();if(i===\"angry\")e.strokeStyle=g,e.lineWidth=2.5;for(let o
of[-1,1])e.beginPath(),e.moveTo(o*(c+5),b-9),e.lineTo(o*(c-3),b-5),e.stroke()}function K(e,i,t)
{e.strokeStyle=g,e.fillStyle=g,e.lineWidth=2.5;let n=9;i===\"happy\"||i===\"loving\"?
(e.beginPath(),e.arc(0,n-2,7,Math.PI*.15,Math.PI*.85),e.stroke()):i===\"sad\"?
(e.beginPath(),e.arc(0,n+7,7,Math.PI*1.2,Math.PI*1.8),e.stroke()):i===\"surprised\"?
(e.beginPath(),e.arc(0,n+1,3.5,0,Math.PI*2),e.stroke()):i===\"scared\"?
(e.beginPath(),e.ellipse(0,n+1,4,5.5,0,0,Math.PI*2),e.fill()):i===\"thinking\"?
(e.beginPath(),e.moveTo(-2,n),e.lineTo(5,n),e.stroke()):i===\"angry\"?
(e.beginPath(),e.arc(0,n+6,6,Math.PI*1.25,Math.PI*1.75),e.stroke()):t?
(e.beginPath(),e.ellipse(0,n,3.5,3,0,0,Math.PI*2),e.fill()):
(e.beginPath(),e.moveTo(-4,n),e.lineTo(4,n),e.stroke())}function _(e,i,t={})
{let{avatar:n,color:s,x:r,y:a,scale:o=1}=i,l=t.mood??
\"\";if(e.save(),e.translate(r,a),e.scale(o,o),Y(e,n,s),e.fillStyle=s,e.strokeStyle=N(s),e.lineWidth=2,e.
beginPath(),e.ellipse(0,0,26,30,0,0,Math.PI*2),e.fill(),e.stroke(),e.fillStyle=\"rgba(255,255,255,0.18)\"
,e.beginPath(),e.ellipse(0,8,15,18,0,0,Math.PI*2),e.fill(),n===\"bird\"&&

```



```

(e.fillStyle="#f59e0b",e.beginPath(),e.moveTo(-5,2),e.lineTo(5,2),e.lineTo(0,9),e.closePath(),e.fill())
,l===\"loving\"){e.fillStyle=\"rgba(244,114,182,0.45)\";for(let d
of[-1,1])e.beginPath(),e.arc(d*15,3,4,0,Math.PI*2),e.fill())G(e,l),n!=\"bird\"&&K(e,l,!t.speaking),e.re
store()var m=720,y=420;function F(e){return e<.5?2*e:e:1-Math.pow(-2*e+2,2)/2}var
S=class{constructor(i,t,n={}){this.sprites=new Map;this.steps=
[];this.stepIndex=0;this.stepElapsed=0;this.moveBase=null;this.sceneSettled=!1;this.conditionalWaitMs=0;t
his.lastTick=0;this.stopped=!1;this.rafId=0;this.path=[];this.finished=!1;this.loop=i=>
{if(this.stopped)return;this.lastTick||(this.lastTick=i);let t=Math.min(i-
this.lastTick,64);this.lastTick=i,this.update(t),this.draw(i),this.rafId=requestAnimationFrame(this.loop)
};this.story=t,this.options=n,this.state=k(t),this.canvas=document.createElement(\"canvas\"),this.canvas.
width=m,this.canvas.height=y,this.canvas.className=\"stage\";let
s=this.canvas.getContext(\"2d\");if(!s)throw new Error(\"Canvas 2D
unavailable\");this.ctx=s,this.choicesPanel=document.createElement(\"div\"),this.choicesPanel.className=\\
\"choices\",this.structurePanel=document.createElement(\"div\"),this.structurePanel.className=\\\"structure\\
\",i.appendChild(this.canvas),i.appendChild(this.choicesPanel),i.appendChild(this.structurePanel),this.ren
derStructure(),this.goToScene(t.initialSceneId),this.rafId=requestAnimationFrame(this.loop)}stop()
{this.stopped=!0,this.rafId&&cancelAnimationFrame(this.rafId)}restart()
{this.state=k(this.story),this.path=[],this.goToScene(this.story.initialSceneId)}sceneExists(i){return
this.story.scenes.some(t=>t.id===i)}goToScene(i){let t=this.story.scenes.find(n=>n.id===i);t&&
(this.scene=t,this.path.push(i),this.finished=!1,this.sprites.clear(),t.characters.forEach((n,s)=>
{this.sprites.set(n.id,
{def:n.x:n.x,y:n.y,visible:!1,bubble:null,phase:s*1.3})),this.steps=I(t.actions),this.stepIndex=0,this.s
tepElapsed=0,this.moveBase=null,this.sceneSettled=!1,this.conditionalWaitMs=0,this.choicesPanel.innerHTML
=\\\"\\\")update(i){for(let t of this.sprites.values())t.bubble&&(t.bubble.remainingMs-
=i,t.bubble.remainingMs<=0&&(t.bubble=null));if(!this.sceneSettled)
{this.advanceProgram(i);return}if(this.scene.conditional&&!this.scene.choices&&
(this.conditionalWaitMs+=i,this.conditionalWaitMs>700)){let
t=E(this.scene.conditional,this.state);this.sceneExists(t)?
this.goToScene(t):this.renderEnd()}advanceProgram(i){if(this.stepIndex>=this.steps.length)
{this.sceneSettled=!0,this.onSettle();return}let t=this.steps[this.stepIndex],n=\"characterId\"in t?
this.sprites.get(t.characterId):void 0;if(t.type===\"appear\"){n&&
(n.visible=!0),this.nextStep();return}if(this.stepElapsed+=i,t.type===\"wait\")
{this.stepElapsed>=t.durationMs&&this.nextStep();return}if(t.type===\"speak\"){n&&(n.visible=!0,n.bubble=
{text:t.text,mood:t.mood??
\\\"\\\",remainingMs:t.durationMs}),this.stepElapsed>=t.durationMs&&this.nextStep();return}if(t.type===\"move
\\\"&&n){this.moveBase||(this.moveBase={x:n.x,y:n.y});let
s=Math.min(this.stepElapsed/t.durationMs,1),r=F(s);n.visible=!0,n.x=this.moveBase.x+t.dx*r,n.y=this.moveB
ase.y+t.dy*r,s>=1&&(this.moveBase=null,this.nextStep())}nextStep()
{this.stepIndex+=1,this.stepElapsed=0}onSettle(){this.scene.choices&&this.scene.choices.length>0?
this.renderChoices():this.scene.conditional||this.renderEnd()}draw(i){let
t=this.ctx,n=B(this.scene.background),s=t.createLinearGradient(0,0,0,y);s.addColorStop(0,n.skyTop),s.addC
olorStop(1,n.skyBottom),t.fillStyle=s,t.fillRect(0,0,m,y),t.fillStyle=n.ground,t.fillRect(0,300,m,y-
300),A(t,this.scene.background);for(let r of this.sprites.values()){if(!r.visible)continue;let
a=Math.sin(i/280+r.phase)*4,o=r.y+a;t.fillStyle=\"rgba(0,0,0,0.12)\\\",t.beginPath(),t.ellipse(r.x,r.y+34,2
6,7,0,0,Math.PI*2),t.fill();let
l=r.def.avatar,d=r.def.color,h=r.def.rule;h&&M(this.state[h.variableId],h.operator,h.value)&&
(l=h.avatar,d=h.color),_(t,{avatar:l,color:d,x:r.x,y:o},
{mood:r.bubble?.mood,speaking:!!r.bubble}),t.font=\"600 13px Inter, system-ui, sans-
serif\\\",t.fillStyle=d,t.textAlign=\"center\\\",t.textBaseline=\"middle\\\",t.fillText(r.def.name,r.x,o+42),r.
bubble&&this.drawBubble(r.x,o-44,r.bubble.text)}this.drawHUD()}drawBubble(i,t,n){let
s=this.ctx;s.font=\"14px Inter, system-ui, sans-serif\\\",s.textAlign=\"left\\\",s.textBaseline=\"top\\\";let
r=this.wrapText(n,230),a=10,o=19,l=Math.min(250,Math.max(...r.map(P=>s.measureText(P).width))+a*2),d=r.le
ngth*a*2,h=i-l/2,v=t-d;h=Math.max(8,Math.min(h,m-l-
8)),v=Math.max(8,v),s.fillStyle=\"rgba(255,255,255,0.97)\\\",s.strokeStyle=\"#c4b5fd\\\",s.lineWidth=2,this.r
oundRect(h,v,l,d,12),s.fill(),s.stroke(),s.fillStyle=\"#1e293b\\\",r.forEach((P,W)=>s.fillText(P,h+a,v+a+W*
o))drawHUD(){let i=this.ctx,t=this.story.variables;if(t.length===0)return;let n=t.map(l=>`$${l.name}:
$${this.state[l.id]}`);i.font=\"600 13px ui-monospace, monospace\\\";let
s=Math.max(...n.map(l=>i.measureText(l).width))+20,r=n.length*20+14,a=m-s-
12,o=12;i.fillStyle=\"rgba(124,58,237,0.10)\\\",i.strokeStyle=\"#7C3AED\\\",i.lineWidth=1.5,this.roundRect(a,
o,s,r,10),i.fill(),i.stroke(),i.fillStyle=\"#5B21B6\\\",i.textAlign=\"left\\\",i.textBaseline=\"top\\\",n.forEa
ch((l,d)=>i.fillText(l,a+10,o+9+d*20))roundRect(i,t,n,s,r)}let
a=this.ctx;a.beginPath(),a.moveTo(i+r,t),a.arcTo(i+n,t,i+n,t+s,r),a.arcTo(i+n,t+s,i,t+s,r),a.arcTo(i,t+s,
i,t,r),a.arcTo(i,t,i+n,t,r),a.closePath()}wrapText(i,t){let n=this.ctx,s=i.split(\" \"),r=
[],a=\\\"\\\";for(let o of s){let l=a+`$${a} $${o}`:o;n.measureText(l).width>t&&a?(r.push(a),a=o):a=l}return
a&&r.push(a),r}renderChoices(){this.choicesPanel.innerHTML=\\\"\\\";for(let i of this.scene.choices??[]){let
t=document.createElement(\"button\\\";t.className=\"choice-btn\\\",t.textContent=i.buttonLabel,t.onclick=
()=>{if(this.sceneExists(i.targetSceneId)){for(let n of
i.modifiers)this.state=C(this.state,n);this.goToScene(i.targetSceneId)}},this.choicesPanel.appendChild(t)
}}renderEnd(){if(this.finished)return;this.finished=!0,this.choicesPanel.innerHTML=\\\"\\\";let
i=document.createElement(\"div\\\";i.className=\"ending\\\",i.textContent=this.options.labels?.end??\"The
End \\\u{1F331}\\\";let t=document.createElement(\"button\\\";t.className=\"choice-btn
secondary\\\",t.textContent=this.options.labels?.playAgain??\"\\u21BA Play again\\\",t.onclick=

```

```
(=>this.restart(),this.choicesPanel.appendChild(i),this.choicesPanel.appendChild(t),this.options.onComplete?.([...this.path],{...this.state}))renderStructure(){let i=R(this.story);this.structurePanel.innerHTML="";let t=(r,a,o)=>`${r} ${r===1?a:o}`;n=[t(i.scenes,"scene","scenes"),t(i.variables,"variable","variables"),t(i.branches,"decision","decisions")],s=this.options.labels?.structure?"Structure: ";this.structurePanel.textContent=`${s}${n.join("\n    \xB7    ")}`;var O=window.__STORY__,L=document.getElementById("stage");O&&L&&new S(L,O,{labels:window.__LABELS__??{}});})();\n";
```

fim de src/generated/standaloneBundle.ts

src/index.ts

Arquivo 17 de 25 · TypeScript · 32 linhas · 561 bytes

SHA-256: 3682fc8b5bc18646e68f6ea307efd5cc92c515a6ef2f588b66f6cb564e571fe6

```
1  // Public entry point of fable-engine.
2
3  export type {
4      Operation,
5      Operator,
6      Value,
7      Variable,
8      Character,
9      Action,
10     Modifier,
11     Choice,
12     Conditional,
13     Scene,
14     Story,
15     State,
16     PrimitiveStep,
17 } from "./engine";
18
19 export {
20     initialState,
21     evaluateConditional,
22     applyModifier,
23     nextSceneByConditional,
24     summarizeAlgorithmicStructure,
25     flatten,
26 } from "./engine";
27
28 export { Player } from "./player";
29 export type { PlayerOptions } from "./player";
30
31 export { exportHtml, downloadHtml } from "./export";
32 export type { ExportLabels } from "./export";
```

fim de src/index.ts

src/moods.ts

Arquivo 18 de 25 · TypeScript · 24 linhas · 828 bytes

SHA-256: 404ccc27448d44d33253ec47c1538497617887a5d0ab7295d9f96ee86ffa7fbf

```
1  // Curated moods/expressions per line of speech. The author picks a line's mood
2  // from a closed set – and the character shows the emotion on its FACE (see
3  // drawCharacter.ts), not as an emoji or as text in the bubble. The mood key maps
4  // directly to the facial expression. Empty = neutral.
5
6  export interface Mood {
7    key: string;
8    name: string;
9  }
10
11 export const MOODS: Mood[] = [
12   { key: "happy", name: "Happy" },
13   { key: "sad", name: "Sad" },
14   { key: "angry", name: "Angry" },
15   { key: "surprised", name: "Surprised" },
16   { key: "scared", name: "Scared" },
17   { key: "thinking", name: "Thinking" },
18   { key: "loving", name: "Loving" },
19 ];
20
21 /** Label of the mood (empty/unknown key → '' = neutral). */
22 export function moodName(key: string | undefined): string {
23   return MOODS.find((m) => m.key === key)?.name ?? "";
24 }
```

fim de src/moods.ts

src/movements.ts

Arquivo 19 de 25 · TypeScript · 50 linhas · 1.148 bytes

SHA-256: d9af275cfe2bc4ede02d3e4c9e147022141d615332a46aabc01599059e08afaf

```
1 // Curated character movements within a scene. The author picks from a closed
2 // menu; each movement becomes one or more `move` steps in the engine. "Repeat
3 // N times" exposes the LOOP (algorithmic thinking) in a controlled way –
4 // expanded by `flatten` in the engine.
5
6 interface Segment {
7   dx: number;
8   dy: number;
9   durationMs: number;
10 }
11
12 export interface Movement {
13   key: string;
14   emoji: string;
15   segments: Segment[];
16 }
17
18 export const MOVEMENTS: Movement[] = [
19   {
20     key: "forward",
21     emoji: "➡",
22     segments: [{ dx: 110, dy: 0, durationMs: 700 }],
23   },
24   {
25     key: "back",
26     emoji: "⬅",
27     segments: [{ dx: -110, dy: 0, durationMs: 700 }],
28   },
29   {
30     key: "jump",
31     emoji: "🦘",
32     segments: [
33       { dx: 0, dy: -55, durationMs: 300 },
34       { dx: 0, dy: 55, durationMs: 300 },
35     ],
36   },
37   {
38     key: "sway",
39     emoji: "↔",
40     segments: [
41       { dx: -24, dy: 0, durationMs: 200 },
42       { dx: 48, dy: 0, durationMs: 300 },
43       { dx: -24, dy: 0, durationMs: 200 },
44     ],
45   },
46 ];
47
48 export function findMovement(key: string): Movement | undefined {
49   return MOVEMENTS.find((m) => m.key === key);
50 }
```

fim de src/movements.ts

src/player.ts

Arquivo 20 de 25 · TypeScript · 430 linhas · 13.213 bytes

SHA-256: 58cfcb3a43a035cab0b7954fcf28c951fd6052fe6b994d2660bf7483c0a4435f

```

1  /**
2   * Animated player – pure Canvas 2D, zero dependencies.
3   *
4   * Runs each scene's animation program (interpolated movement, speech bubbles),
5   * keeps variable state (HUD), handles choices (branching) and conditionals
6   * (state-based branching).
7   *
8   * – ``stop()`` ends the animation loop (cleanup when unmounting);
9   * – it tracks the path (sequence of scenes) and, on reaching an ending,
10  *   calls ``onComplete(path, finalState)``;
11  * – interactive transitions tolerate a missing target (story under
12  *   construction): a choice without a valid target is ignored instead of breaking.
13  */
14
15  import {
16    applyModifier,
17    summarizeAlgorithmicStructure,
18    evaluateConditional,
19    flatten,
20    initialState,
21    nextSceneByConditional,
22    type Character,
23    type Story,
24    type PrimitiveStep,
25    type Scene,
26    type State,
27  } from "../engine";
28  import { resolveBackground } from "../backgrounds";
29  import { drawBackground } from "../drawBackground";
30  import { drawCharacter } from "../drawCharacter";
31
32  interface SpriteRuntime {
33    def: Character;
34    x: number;
35    y: number;
36    visible: boolean;
37    bubble: { text: string; mood: string; remainingMs: number } | null;
38    phase: number;
39  }
40
41  export interface PlayerOptions {
42    /** Called on reaching an ending (a scene with no choices and no conditional). */
43    onComplete?: (path: string[], finalState: State) => void;
44    /** UI labels (i18n). */
45    labels?: { end?: string; playAgain?: string; structure?: string };
46  }
47
48  const WIDTH = 720;
49  const HEIGHT = 420;
50
51  function easeInOutQuad(t: number): number {
52    return t < 0.5 ? 2 * t * t : 1 - Math.pow(-2 * t + 2, 2) / 2;
53  }
54
55  export class Player {
56    private canvas: HTMLCanvasElement;
57    private ctx: CanvasRenderingContext2D;
58    private choicesPanel: HTMLElement;
59    private structurePanel: HTMLElement;
60
61    private story: Story;
62    private state: State;
63    private scene!: Scene;
64    private sprites = new Map<string, SpriteRuntime>();
65
66    private steps: PrimitiveStep[] = [];
67    private stepIndex = 0;

```

```
68     private stepElapsed = 0;
69     private moveBase: { x: number; y: number } | null = null;
70     private sceneSettled = false;
71     private conditionalWaitMs = 0;
72
73     private lastTick = 0;
74     private stopped = false;
75     private rafId = 0;
76     private path: string[] = [];
77     private finished = false;
78     private options: PlayerOptions;
79
80     constructor(container: HTMLElement, story: Story, options: PlayerOptions = {}) {
81         this.story = story;
82         this.options = options;
83         this.state = initialState(story);
84
85         this.canvas = document.createElement("canvas");
86         this.canvas.width = WIDTH;
87         this.canvas.height = HEIGHT;
88         this.canvas.className = "stage";
89         const ctx = this.canvas.getContext("2d");
90         if (!ctx) throw new Error("Canvas 2D unavailable");
91         this.ctx = ctx;
92
93         this.choicesPanel = document.createElement("div");
94         this.choicesPanel.className = "choices";
95         this.structurePanel = document.createElement("div");
96         this.structurePanel.className = "structure";
97
98         container.appendChild(this.canvas);
99         container.appendChild(this.choicesPanel);
100        container.appendChild(this.structurePanel);
101
102        this.renderStructure();
103        this.goToScene(story.initialSceneId);
104        this.rafId = requestAnimationFrame(this.loop);
105    }
106
107    /** Ends the animation loop (cleanup when unmounting). */
108    stop(): void {
109        this.stopped = true;
110        if (this.rafId) cancelAnimationFrame(this.rafId);
111    }
112
113    restart(): void {
114        this.state = initialState(this.story);
115        this.path = [];
116        this.goToScene(this.story.initialSceneId);
117    }
118
119    private sceneExists(id: string): boolean {
120        return this.story.scenes.some((s) => s.id === id);
121    }
122
123    private goToScene(id: string): void {
124        const scene = this.story.scenes.find((s) => s.id === id);
125        if (!scene) return; // tolerate a missing target (story under construction)
126        this.scene = scene;
127        this.path.push(id);
128        this.finished = false; // new scene: the ending has not been reached yet
129
130        this.sprites.clear();
131        scene.characters.forEach((c, i) => {
132            this.sprites.set(c.id, {
133                def: c,
134                x: c.x,
135                y: c.y,
136                visible: false,
137                bubble: null,
138                phase: i * 1.3,
139            });
140        });
141    }
```

```

140     });
141
142     this.steps = flatten(scene.actions);
143     this.stepIndex = 0;
144     this.stepElapsed = 0;
145     this.moveBase = null;
146     this.sceneSettled = false;
147     this.conditionalWaitMs = 0;
148     this.choicesPanel.innerHTML = "";
149 }
150
151 private loop = (t: number): void => {
152     if (this.stopped) return;
153     if (!this.lastTick) this.lastTick = t;
154     const dt = Math.min(t - this.lastTick, 64);
155     this.lastTick = t;
156
157     this.update(dt);
158     this.draw(t);
159     this rafId = requestAnimationFrame(this.loop);
160 };
161
162 private update(dt: number): void {
163     for (const s of this.sprites.values()) {
164         if (s.bubble) {
165             s.bubble.remainingMs -= dt;
166             if (s.bubble.remainingMs <= 0) s.bubble = null;
167         }
168     }
169
170     if (!this.sceneSettled) {
171         this.advanceProgram(dt);
172         return;
173     }
174
175     if (this.scene.conditional && !this.scene.choices) {
176         this.conditionalWaitMs += dt;
177         if (this.conditionalWaitMs > 700) {
178             const target = nextSceneByConditional(this.scene.conditional, this.state);
179             if (this.sceneExists(target)) this.goToScene(target);
180             else this.renderEnd(); // empty target → end instead of freezing
181         }
182     }
183 }
184
185 private advanceProgram(dt: number): void {
186     if (this.stepIndex >= this.steps.length) {
187         this.sceneSettled = true;
188         this.onSettle();
189         return;
190     }
191
192     const step = this.steps[this.stepIndex];
193     const sprite =
194         "characterId" in step ? this.sprites.get(step.characterId) : undefined;
195
196     if (step.type === "appear") {
197         if (sprite) sprite.visible = true;
198         this.nextStep();
199         return;
200     }
201
202     this.stepElapsed += dt;
203
204     if (step.type === "wait") {
205         if (this.stepElapsed >= step.durationMs) this.nextStep();
206         return;
207     }
208
209     if (step.type === "speak") {
210         if (sprite) {
211             sprite.visible = true;

```



```

212         sprite.bubble = {
213             text: step.text,
214             mood: step.mood ?? "",
215             remainingMs: step.durationMs,
216         };
217     }
218     if (this.stepElapsed >= step.durationMs) this.nextStep();
219     return;
220 }
221
222 if (step.type === "move" && sprite) {
223     if (!this.moveBase) this.moveBase = { x: sprite.x, y: sprite.y };
224     const p = Math.min(this.stepElapsed / step.durationMs, 1);
225     const e = easeInOutQuad(p);
226     sprite.visible = true;
227     sprite.x = this.moveBase.x + step.dx * e;
228     sprite.y = this.moveBase.y + step.dy * e;
229     if (p >= 1) {
230         this.moveBase = null;
231         this.nextStep();
232     }
233 }
234 }
235
236 private nextStep(): void {
237     this.stepIndex += 1;
238     this.stepElapsed = 0;
239 }
240
241 private onSettle(): void {
242     if (this.scene.choices && this.scene.choices.length > 0) {
243         this.renderChoices();
244     } else if (!this.scene.conditional) {
245         this.renderEnd();
246     }
247 }
248
249 private draw(t: number): void {
250     const ctx = this.ctx;
251     // Curated background: sky + ground colors (no images).
252     const background = resolveBackground(this.scene.background);
253     const sky = ctx.createLinearGradient(0, 0, 0, HEIGHT);
254     sky.addColorStop(0, background.skyTop);
255     sky.addColorStop(1, background.skyBottom);
256     ctx.fillStyle = sky;
257     ctx.fillRect(0, 0, WIDTH, HEIGHT);
258     ctx.fillStyle = background.ground;
259     ctx.fillRect(0, 300, WIDTH, HEIGHT - 300);
260     // Background elements (trees, sun, buildings...) drawn on the Canvas.
261     drawBackground(ctx, this.scene.background);
262
263     for (const s of this.sprites.values()) {
264         if (!s.visible) continue;
265         const bob = Math.sin(t / 280 + s.phase) * 4;
266         const cy = s.y + bob;
267
268         ctx.fillStyle = "rgba(0,0,0,0.12)";
269         ctx.beginPath();
270         ctx.ellipse(s.x, s.y + 34, 26, 7, 0, 0, Math.PI * 2);
271         ctx.fill();
272
273         // "Visual reacts to a variable": if the character's rule holds in the
274         // current state, use the alternate (curated) avatar.
275         let avatar = s.def.avatar;
276         let color = s.def.color;
277         const r = s.def.rule;
278         if (r && evaluateConditional(this.state[r.variableId], r.operator, r.value)) {
279             avatar = r.avatar;
280             color = r.color;
281         }
282
283         // Drawn character: the face carries the emotion of the current line

```

```

284 // (mood) – no emoji, no emotion bubble. Speaking opens the mouth slightly.
285 drawCharacter(
286   ctx,
287   { avatar, color, x: s.x, y: cy },
288   { mood: s.bubble?.mood, speaking: !!s.bubble },
289 );
290
291 ctx.font = "600 13px Inter, system-ui, sans-serif";
292 ctx.fillStyle = color;
293 ctx.textAlign = "center";
294 ctx.textBaseline = "middle";
295 ctx.fillText(s.def.name, s.x, cy + 42);
296
297 if (s.bubble) this.drawBubble(s.x, cy - 44, s.bubble.text);
298 }
299
300 this.drawHUD();
301 }
302
303 private drawBubble(cx: number, cy: number, text: string): void {
304   const ctx = this.ctx;
305   ctx.font = "14px Inter, system-ui, sans-serif";
306   ctx.textAlign = "left";
307   ctx.textBaseline = "top";
308   const lines = this.wrapText(text, 230);
309   const pad = 10;
310   const lh = 19;
311   const w = Math.min(
312     250,
313     Math.max(...lines.map((l) => ctx.measureText(l).width)) + pad * 2,
314   );
315   const h = lines.length * lh + pad * 2;
316   let x = cx - w / 2;
317   let y = cy - h;
318   x = Math.max(8, Math.min(x, WIDTH - w - 8));
319   y = Math.max(8, y);
320
321   ctx.fillStyle = "rgba(255,255,255,0.97)";
322   ctx.strokeStyle = "#c4b5fd";
323   ctx.lineWidth = 2;
324   this.roundRect(x, y, w, h, 12);
325   ctx.fill();
326   ctx.stroke();
327
328   ctx.fillStyle = "#1e293b";
329   lines.forEach((l, i) => ctx.fillText(l, x + pad, y + pad + i * lh));
330 }
331
332 private drawHUD(): void {
333   const ctx = this.ctx;
334   const vars = this.story.variables;
335   if (vars.length === 0) return;
336   const lines = vars.map((v) => `${v.name}: ${this.state[v.id]}`);
337   ctx.font = "600 13px ui-monospace, monospace";
338   const w = Math.max(...lines.map((l) => ctx.measureText(l).width)) + 20;
339   const h = lines.length * 20 + 14;
340   const x = WIDTH - w - 12;
341   const y = 12;
342   ctx.fillStyle = "rgba(124,58,237,0.10)";
343   ctx.strokeStyle = "#7C3AED";
344   ctx.lineWidth = 1.5;
345   this.roundRect(x, y, w, h, 10);
346   ctx.fill();
347   ctx.stroke();
348   ctx.fillStyle = "#5B21B6";
349   ctx.textAlign = "left";
350   ctx.textBaseline = "top";
351   lines.forEach((l, i) => ctx.fillText(l, x + 10, y + 9 + i * 20));
352 }
353
354 private roundRect(x: number, y: number, w: number, h: number, r: number): void {
355   const ctx = this.ctx;

```

```

356     ctx.beginPath();
357     ctx.moveTo(x + r, y);
358     ctx.arcTo(x + w, y, x + w, y + h, r);
359     ctx.arcTo(x + w, y + h, x, y + h, r);
360     ctx.arcTo(x, y + h, x, y, r);
361     ctx.arcTo(x, y, x + w, y, r);
362     ctx.closePath();
363 }
364
365 private wrapText(text: string, maxWidth: number): string[] {
366     const ctx = this.ctx;
367     const words = text.split(" ");
368     const lines: string[] = [];
369     let current = "";
370     for (const word of words) {
371         const attempt = current ? `${current} ${word}` : word;
372         if (ctx.measureText(attempt).width > maxWidth && current) {
373             lines.push(current);
374             current = word;
375         } else {
376             current = attempt;
377         }
378     }
379     if (current) lines.push(current);
380     return lines;
381 }
382
383 private renderChoices(): void {
384     this.choicesPanel.innerHTML = "";
385     for (const choice of this.scene.choices ?? []) {
386         const button = document.createElement("button");
387         button.className = "choice-btn";
388         button.textContent = choice.buttonLabel;
389         button.onclick = () => {
390             if (!this.sceneExists(choice.targetSceneId)) return; // tolerate unfinished target
391             for (const mod of choice.modifiers)
392                 this.state = applyModifier(this.state, mod);
393             this.goToScene(choice.targetSceneId);
394         };
395         this.choicesPanel.appendChild(button);
396     }
397 }
398
399 private renderEnd(): void {
400     // Guard against re-entrancy: a conditional branch without a valid target
401     // re-evaluates every frame; without this, the ending would fire ~60x/s.
402     if (this.finished) return;
403     this.finished = true;
404     this.choicesPanel.innerHTML = "";
405     const ending = document.createElement("div");
406     ending.className = "ending";
407     ending.textContent = this.options.labels?.end ?? "The End 🌱";
408     const restart = document.createElement("button");
409     restart.className = "choice-btn secondary";
410     restart.textContent = this.options.labels?.playAgain ?? "🔄 Play again";
411     restart.onclick = () => this.restart();
412     this.choicesPanel.appendChild(ending);
413     this.choicesPanel.appendChild(restart);
414     this.options.onComplete?.([...this.path], { ...this.state });
415 }
416
417 private renderStructure(): void {
418     const s = summarizeAlgorithmicStructure(this.story);
419     this.structurePanel.innerHTML = "";
420     const plural = (n: number, one: string, many: string): string =>
421         `${n} ${n === 1 ? one : many}`;
422     const items = [
423         plural(s.scenes, "scene", "scenes"),
424         plural(s.variables, "variable", "variables"),
425         plural(s.branches, "decision", "decisions"),
426     ];
427     const label = this.options.labels?.structure ?? "Structure:";

```

```
428 |         this.structurePanel.textContent = `${label} ${items.join(" · ")}`;
429 |     }
430 | }
```

fim de src/player.ts

src/standalone.ts

Arquivo 21 de 25 · TypeScript · 22 linhas · 782 bytes

SHA-256: 0326f717e51f40d040d6a16d4a310fc28452ede88cb17cd67852c56e7f7030c1

```
1  // Standalone player entry for the self-contained "file over app" export.
2  //
3  // This module is bundled by esbuild into a single IIFE (no external imports) and
4  // embedded in the self-contained .html the reader takes with them. It reads the
5  // `Story` from `window.__STORY__` (injected into the HTML) and mounts the
6  // player on #stage. Zero network requests – opens offline in any browser.
7
8  import type { Story } from "./engine";
9  import { Player } from "./player";
10
11 declare global {
12   interface Window {
13     __STORY__: Story;
14     __LABELS__: { end?: string; playAgain?: string; structure?: string };
15   }
16 }
17
18 const story = window.__STORY__;
19 const stage = document.getElementById("stage");
20 if (story && stage) {
21   new Player(stage, story, { labels: window.__LABELS__ ?? {} });
22 }
```

fim de src/standalone.ts

test/e2e.mjs

Arquivo 22 de 25 · JavaScript (ESM) · 135 linhas · 4.435 bytes

SHA-256: ca7b77adea73fb72e593ac3cb6ab140a820f7ad3e8c8976e00e46cfc06769033

```
1 // End-to-end reproduction of the headline claim: the exported self-contained
2 // .html plays offline and makes ZERO external network requests.
3 //
4 // Run: `npm run test:e2e`
5 //
6 // What it does (no framework, like test/selfcheck):
7 // 1. Regenerates the inlined player bundle and produces the export HTML for
8 //    `foxAndWolf` – without hardcoding – by bundling a tiny entry that imports
9 //    `exportHtml` and the example story, then writing the HTML to a temp file.
10 // 2. Launches headless chromium and blocks every non-local scheme, recording
11 //    any http/https/ws request as an "external request".
12 // 3. Loads the file, checks the canvas rendered, plays through to an ending,
13 //    and asserts zero external requests were made.
14
15 import { build } from "esbuild";
16 import { execFileSync } from "node:child_process";
17 import { mkdtempSync, rmSync } from "node:fs";
18 import { tmpdir } from "node:os";
19 import { fileURLToPath, pathToFileURL } from "node:url";
20 import { dirname, resolve, join } from "node:path";
21 import { chromium } from "playwright";
22
23 const root = resolve(dirname(fileURLToPath(import.meta.url)), "..");
24 const dir = mkdtempSync(join(tmpdir(), "fable-e2e-"));
25 const htmlPath = join(dir, "story.html");
26
27 let ok = 0;
28 let fail = 0;
29 function check(name, cond) {
30   if (cond) {
31     ok += 1;
32   } else {
33     fail += 1;
34     console.error(` x ${name}`);
35   }
36 }
37
38 async function generateHtml() {
39   // Make sure the inlined player runtime is fresh.
40   execFileSync("npm", ["run", "build:standalone"], { cwd: root, stdio: "inherit" });
41
42   // Bundle a tiny entry that produces the export HTML from the real source,
43   // so nothing here is hardcoded.
44   const entry = join(dir, "make-html.mjs");
45   await build({
46     stdin: {
47       contents: `
48         import { exportHtml } from "./src/export";
49         import { foxAndWolf } from "./examples/fox-and-wolf";
50         import { writeFileSync } from "node:fs";
51         writeFileSync(process.argv[2], exportHtml(foxAndWolf));
52       `,
53       resolveDir: root,
54       loader: "ts",
55     },
56     bundle: true,
57     format: "esm",
58     platform: "node",
59     target: "es2020",
60     outfile: entry,
61   });
62   execFileSync("node", [entry, htmlPath], { stdio: "inherit" });
63 }
64
65 async function main() {
66   await generateHtml();
67 }
```

```

68   const browser = await chromium.launch();
69   const context = await browser.newContext();
70   const external = [];
71
72   // Allow only local schemes; record + abort anything that would hit a network.
73   await context.route("**", (route) => {
74     const url = route.request().url();
75     if (/^(file|data|blob):/i.test(url)) {
76       route.continue();
77     } else {
78       external.push(url);
79       route.abort();
80     }
81   });
82
83   const page = await context.newPage();
84   await page.goto(pathToFileURL(htmlPath).href);
85
86   // (1) canvas rendered
87   const canvas = page.locator("canvas.stage");
88   await canvas.waitFor({ state: "attached", timeout: 5000 });
89   const width = await canvas.evaluate((el) => el.width);
90   check("canvas rendered (exists, width > 0)", width > 0);
91
92   // (2) playable to an ending: play scenes (which animate in real time), clicking
93   // a real choice button whenever one appears, until an `.ending` shows up.
94   // Timeouts are generous because scenes animate over several seconds.
95   const STEP_TIMEOUT = 20000;
96   const ending = page.locator(".ending");
97   const choice = page.locator(".choice-btn:not(.secondary)");
98   let reachedEnding = false;
99   for (let i = 0; i < 6; i += 1) {
100     // Wait for the scene to settle into either an ending or a choice.
101     try {
102       await page
103         .locator(".ending, .choice-btn:not(.secondary)")
104         .first()
105         .waitFor({ state: "visible", timeout: STEP_TIMEOUT });
106     } catch {
107       break; // nothing actionable appeared -> will fail below
108     }
109     if ((await ending.count()) > 0) {
110       reachedEnding = true;
111       break;
112     }
113     await choice.first().click();
114   }
115   check("story is playable to an ending", reachedEnding);
116
117   // (3) zero external requests during the whole run
118   if (external.length > 0) {
119     console.error(` external requests: ${external.join(", ")}`);
120   }
121   check("zero external network requests", external.length === 0);
122
123   await browser.close();
124 }
125
126 main()
127   .catch((err) => {
128     fail += 1;
129     console.error(err);
130   })
131   .finally(() => {
132     rmSync(dir, { recursive: true, force: true });
133     console.log(`\ne2e: ${ok} passed, ${fail} failed`);
134     process.exit(fail > 0 ? 1 : 0);
135   });

```

fim de test/e2e.mjs

test/selfcheck.ts

Arquivo 23 de 25 · TypeScript · 92 linhas · 2.882 bytes

SHA-256: 6bc02d416c5e168e7015cfa680a5d52e7cdd28d0e0b4e8ae64f862b5f6dc87a0

```
1  /**
2   * Self-check of the engine's core logic (tests-as-specs).
3   * Runs headless: `npm test`. No framework, no browser.
4   */
5
6  import {
7    applyModifier,
8    summarizeAlgorithmicStructure,
9    evaluateConditional,
10   flatten,
11   initialState,
12   nextSceneByConditional,
13   type State,
14 } from "../src/engine";
15 import { foxAndWolf } from "../examples/fox-and-wolf";
16
17 let ok = 0;
18 let fail = 0;
19
20 function check(name: string, cond: boolean): void {
21   if (cond) {
22     ok += 1;
23   } else {
24     fail += 1;
25     console.error(` x ${name}`);
26   }
27 }
28
29 // --- evaluateConditional ---
30 check("courage 2 >= 2", evaluateConditional(2, ">=", 2) === true);
31 check("courage 1 >= 2 false", evaluateConditional(1, ">=", 2) === false);
32 check("3 > 2", evaluateConditional(3, ">", 2) === true);
33 check("2 < 2 false", evaluateConditional(2, "<", 2) === false);
34 check("== equality", evaluateConditional(5, "==", 5) === true);
35 check("!= difference", evaluateConditional(5, "!=", 4) === true);
36 check("boolean == true", evaluateConditional(true, "==", true) === true);
37 check("boolean == false", evaluateConditional(true, "==", false) === false);
38
39 // --- applyModifier (immutable) ---
40 const base: State = { courage: 0 };
41 check(
42   "add +2",
43   applyModifier(base, { variableId: "courage", operation: "+", value: 2 })
44     .courage === 2,
45 );
46 check(
47   "subtract -1",
48   applyModifier(base, { variableId: "courage", operation: "-", value: 1 })
49     .courage === -1,
50 );
51 check(
52   "assign =",
53   applyModifier(base, { variableId: "courage", operation: "=", value: 9 })
54     .courage === 9,
55 );
56 check("does not mutate original", base.courage === 0);
57
58 // --- flatten (expands repeat) ---
59 const steps = flatten(foxAndWolf.scenes[0].actions);
60 check("flatten expands loop 3x -> 9 steps", steps.length === 9);
61 check(
62   "flatten only yields primitive steps",
63   steps.every((s) => ["appear", "speak", "move", "wait"].includes(s.type)),
64 );
65
66 // --- summarizeAlgorithmicStructure ---
67 const c = summarizeAlgorithmicStructure(foxAndWolf);
```



```
68 | check("structure: scenes = 4", c.scenes === 4);
69 | check("structure: variables = 1", c.variables === 1);
70 | check(
71 |   "structure: branches = 2 choices + 1 conditional = 3",
72 |   c.branches === 3,
73 | );
74 | check("structure: modifiers = 2", c.modifiers === 2);
75 |
76 | // --- symbolic playthrough of both branches ---
77 | function run(courageChoice: number): string {
78 |   let state = initialState(foxAndWolf);
79 |   state = applyModifier(state, {
80 |     variableId: "courage",
81 |     operation: "+",
82 |     value: courageChoice,
83 |   });
84 |   const c2 = foxAndWolf.scenes.find((x) => x.id === "c2");
85 |   if (!c2?.conditional) throw new Error("c2 has no conditional");
86 |   return nextSceneByConditional(c2.conditional, state);
87 | }
88 | check("brave choice (+2) -> victory", run(2) === "c_victory");
89 | check("timid choice (-1) -> flight", run(-1) === "c_flight");
90 |
91 | console.log(`\nself-check: ${ok} passed, ${fail} failed`);
92 | if (fail > 0) throw new Error(`${fail} check(s) failed`);
```

fim de test/selfcheck.ts

tsconfig.build.json

Arquivo 24 de 25 · JSON · 8 linhas · 128 bytes

SHA-256: a42408a1fe307dae1330403900eed592a1d100893d0011a2197773df677b631e

```
1  | {
2  |   "extends": "./tsconfig.json",
3  |   "compilerOptions": {
4  |     "rootDir": "src",
5  |     "outDir": "dist"
6  |   },
7  |   "include": ["src"]
8  | }
```

fim de tsconfig.build.json

tsconfig.json

Arquivo 25 de 25 · JSON · 18 linhas · 444 bytes

SHA-256: 94c33ce16453a0c80d8c3b806660d50d96c04ea2650deeaeb2d47b06e8103e6b

```
1  {
2    "compilerOptions": {
3      "target": "ES2020",
4      "module": "ESNext",
5      "moduleResolution": "Bundler",
6      "lib": ["ES2020", "DOM"],
7      "declaration": true,
8      "strict": true,
9      "noUnusedLocals": true,
10     "noUnusedParameters": true,
11     "noImplicitReturns": true,
12     "esModuleInterop": true,
13     "forceConsistentCasingInFileNames": true,
14     "skipLibCheck": true,
15     "outDir": "dist"
16   },
17   "include": ["src", "test", "examples"]
18 }
```

fim de tsconfig.json